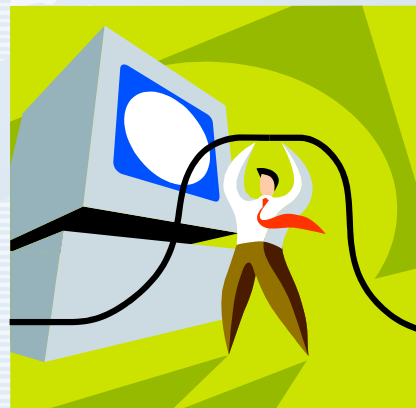


第四章 动态规划



学习要点

理解动态规划算法的概念。

掌握动态规划算法的基本要素

(1) 最优子结构性质

(2) 重叠子问题

理解动态规划算法与贪心算法的差异

理解动态规划算法的一般理论

通过应用范例学习动态规划设计策略。

- 什么是动态规划
- 动态规划与多段图
- 资源分配问题
- 0/1背包问题
- 可靠性设计
- 货郎担问题
- 流水线调度问题

动态规划是解决最优化问题的方法之一。

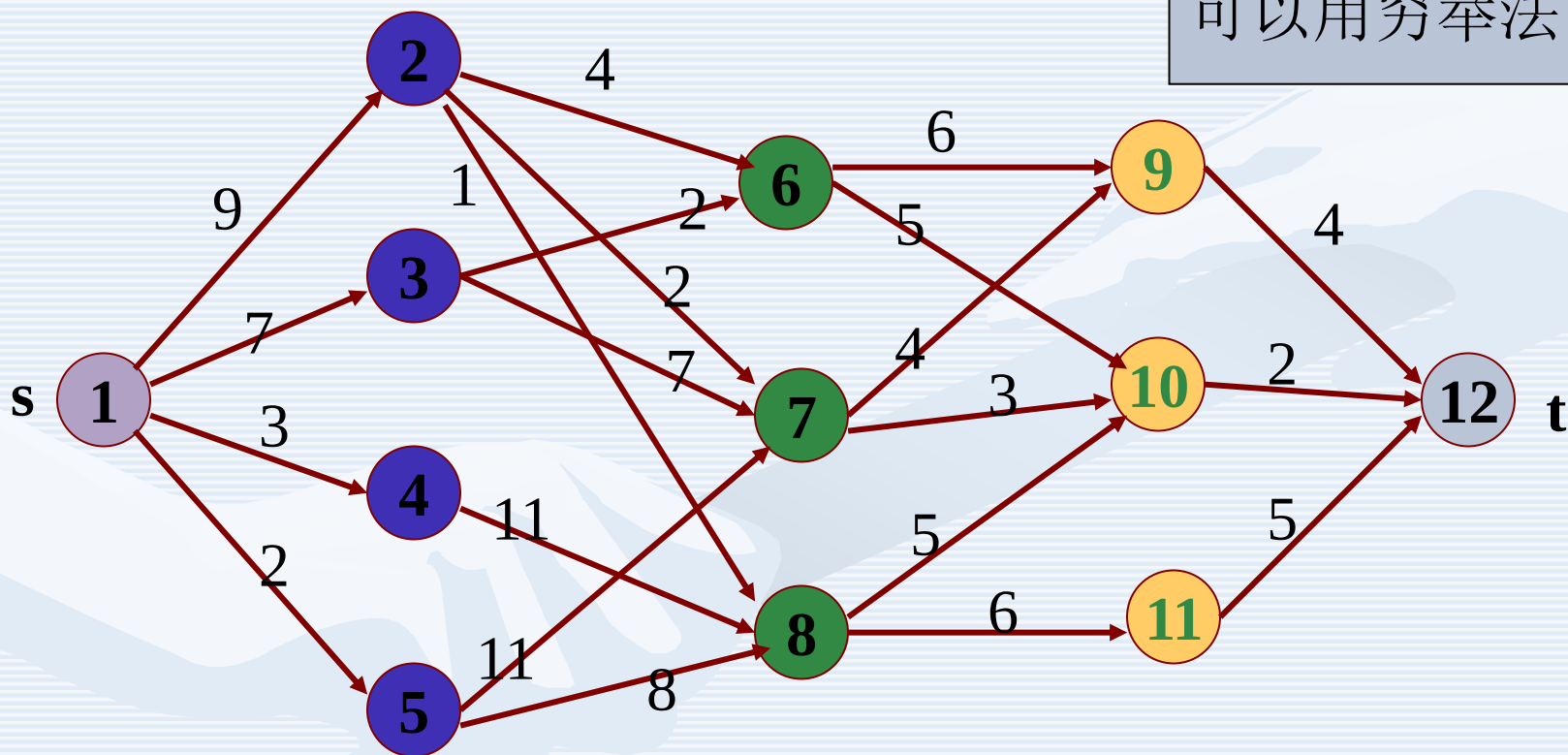
动态规划是现代管理中一种重要的决策方法,它可以广泛地用于解决**最短路径问题、资源分配问题、生产计划与库存问题、投资问题、装载问题、排序问题及生产过程的最优控制**等。由于它具有独特的解题思路因此在处理某些优化问题时常比线性规划等方法更为有效。

问题的提出

首先，例举一个典型的且很直观的决策问题：

[例] 下图表示城市之间的交通路网，线段上的数字表示费用，单向通行由s->t。求s->t最省的费用。

采用的求解方法，
可以用穷举法。



从上例的求解中，我们不仅得到由s点出发到终点t的最短路线及最短距离，而且还得到了从所有各中间点到终点的最短路线及最短距离，这对许多实际问题来讲是很有用的。

与穷举法相比，动态规划的方法有两个明显的优点：

(1)大大减少了计算量

(2)丰富了计算结果

动态规划的最优化概念是在一定条件下，找到一种途径，在对各阶段的效益经过按问题具体性质所确定的运算以后，使得全过程的总效益达到最优。

应用动态规划要注意阶段的划分是关键，必须依据题意分析，寻求合理的划分阶段(子问题)方法。而每个子问题是一个比原问题简单得多的优化问题。而且每个子问题的求解中，均利用它的一个后部子问题的最优化结果，直到最后一个子问题所得最优解，它就是原问题的最优解。

显然，用枚举的方法从所有可能的决策序列中去选取最优决策序列是一种最笨拙的方法。

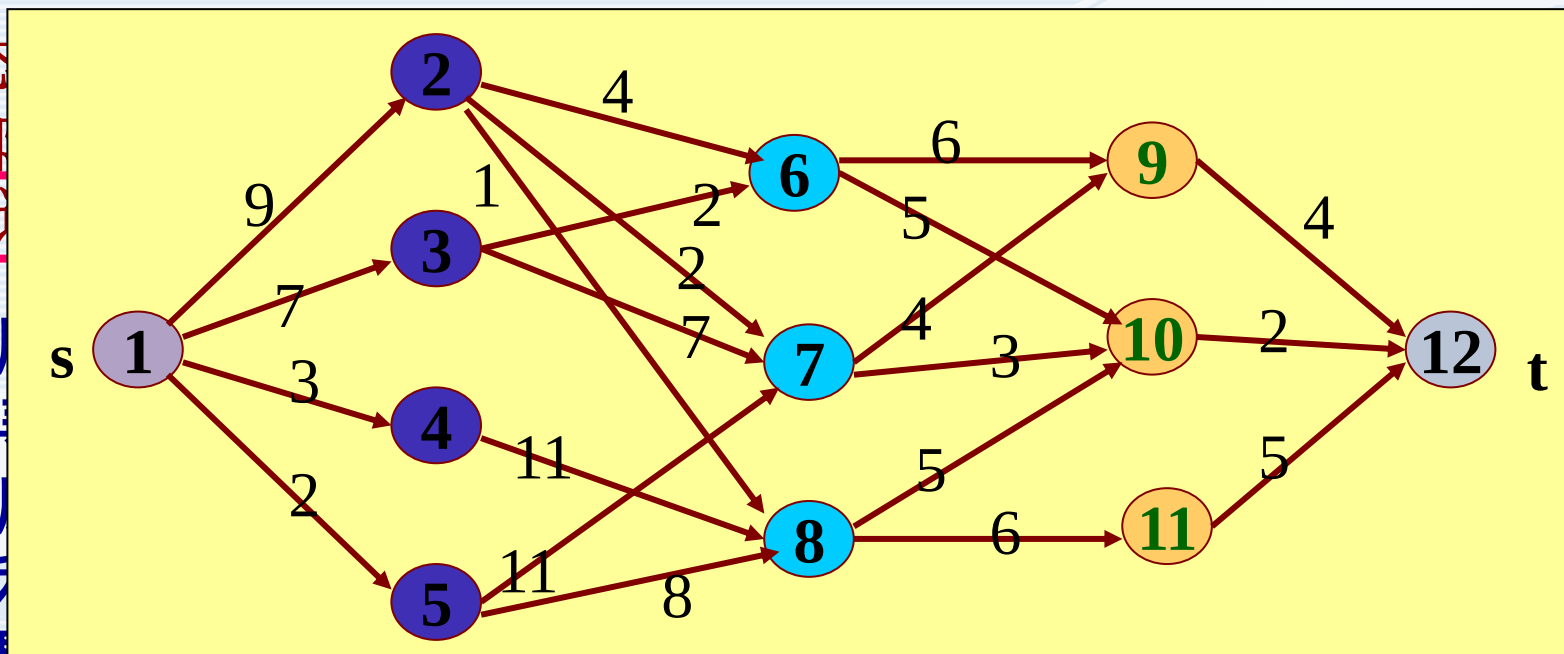
采用动态规划法，常常能得到一个比枚举法有效的算法

与穷举法相比，动态规划的方法有两个明显的优点：

(1)大大减少了计算量

(2)丰富了计算结果

动态
在
使得全
应
分析，
一个比
中，均
个子问



题所求得最优解，也就是原问题的最优解。

4.1 一般方法

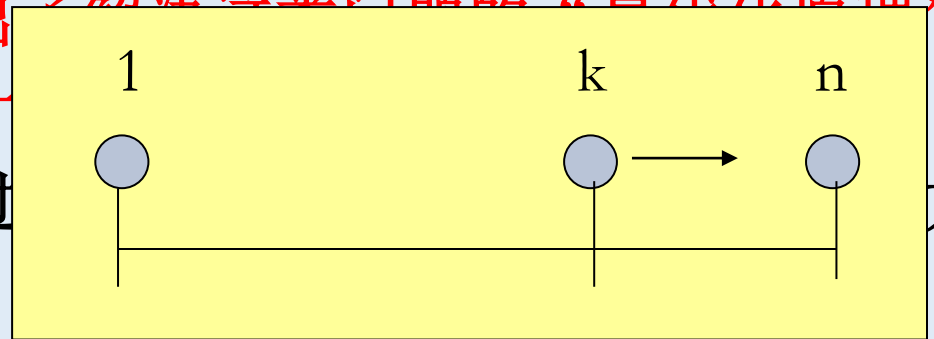
4.1.1 一般概念

- 在实际生活中，有一类问题的活动过程可以分为若干个阶段，而且在任一阶段后的行为都依赖于该阶段的状态，而与该阶段之前的过程是如何达到这种状态的方式无关，这类问题的解决就构成了一个多阶段决策过程。
- 在50年代，贝尔曼等人提出了解决这类问题的“最优化原理”，从而创建了最优化问题的一种新的算法设计——动态规划。
- 动态规划方法是处理分段过程最优化问题的一类及其有效的方法。
- 最优性原理(Principle of Optimality)
多阶段过程的最优决策序列应当具有如下性质：
 - ◆ 无论过程的初始状态和初始决策是什么，其余的决策都必须相对于初始决策所产生的状态构成一个最优决策序列。

4.1 一般方法

4.1.1 一般概念

- 在实际生活中，有一类问题的活动过程可以分为若干个阶段，而且在任一阶段后的行为都依赖于该阶段的状态，而与该阶段之前的过程是如何达到这种状态的方式无关，这类问题的解决就构成了一个多阶段决策过程。
- 在50年代，贝尔曼等人提出了解决这类问题的“最优化原理”，从而创建了最优化问题的一
- 动态规划方法是处理分段过



- 最优性原理(Principle of Optimality)

多阶段过程的最优决策序列应当具有如下性质：

◆无论过程的初始状态和初始决策是什么，其余的决策都必须相对于初始决策所产生的状态构成一个最优决策序列。

- 最优化原理是动态规划的基础，任何问题，如果失去了最优化原理的支持，就不可能用动态规划方法计算。动态规划的最优化理论在其指标函数的可分离性和单调性中得到体现。根据最优化原理导出的动态规划基本方程是解决一切动态规划问题的基本方法。

- 如果对于所求解的问题最优性原理成立；则说明用动态规划方法有可能解决该问题。而解决问题的关键在于获取各阶段间的递推关系式。

- 任何多阶段过程问题，只有证明其最优化原理成立，才可能用动态规划的方法予以解决。

④ 在多阶段决策过程的每一个阶段，都可能有多种选择的决策，但必须从中选取一种决策。一旦各种阶段的决策选定之后，就构成了解决这一问题的一个决策序列。决策序列不同，导致的问题结果也不同。

④ 所以动态规划的目标就是要在所有容许选择的决策序列中选取一个会获得问题最优解的决策序列，即**最优决策序列**。

◆动态规划算法的适用条件

动态规划不是万能的，它只适于解决**一定条件的**最优策略问题。

很多问题都可以用动态规划法来求解，因为这个范围内的问题极多，还有许多看似不是这个范围中的问题都可以转化成这类问题。

而问题的最优子结构性性质和子问题重叠性质是采用动态规划算法的两个基本要素。

- 对于一个多阶段过程问题，上述最优决策是否存在,依赖于该问题是否有最优子结构性性质,而能否采用动态规划的方法，还要看该问题的子问题是否具有重叠性质。

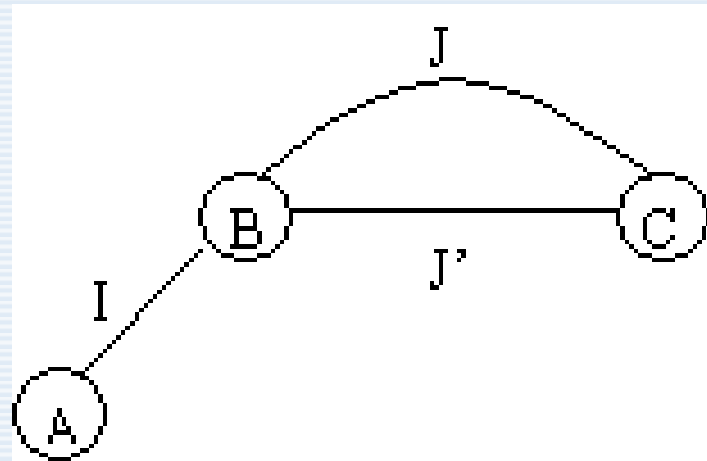
最优子结构性性质：原问题的最优解包含了其子问题的最优解。——满足最优性原理

子问题重叠性质：每次产生的子问题并不总是新问题，有些子问题被反复计算多次。

1. 最优子结构性质：

在分析问题的最优子结构性质时，所用的方法具有普遍性：首先假设由问题的最优解导出的子问题的解不是最优的，然后再设法说明在这个假设下可构造出比原问题最优解更好的解，从而导致矛盾。

例如图中，若路线**I**和**J**是**A**到**C**的最优路径，则根据最优化原理，路线**J**必是从**B**到**C**的最优路线。这可用反证法证明：假设有另一路径**J'**是**B**到**C**的最优路径，则**A**到**C**的路线取**I**和**J'**比**I**和**J**更优，矛盾。从而证明**J'**必是**B**到**C**的最优路径。



——利用问题的最优子结构性质，以自底向上的方式递归地从子问题的最优解逐步构造出整个问题的最优解。最优子结构是问题能用动态规划算法求解的前提。

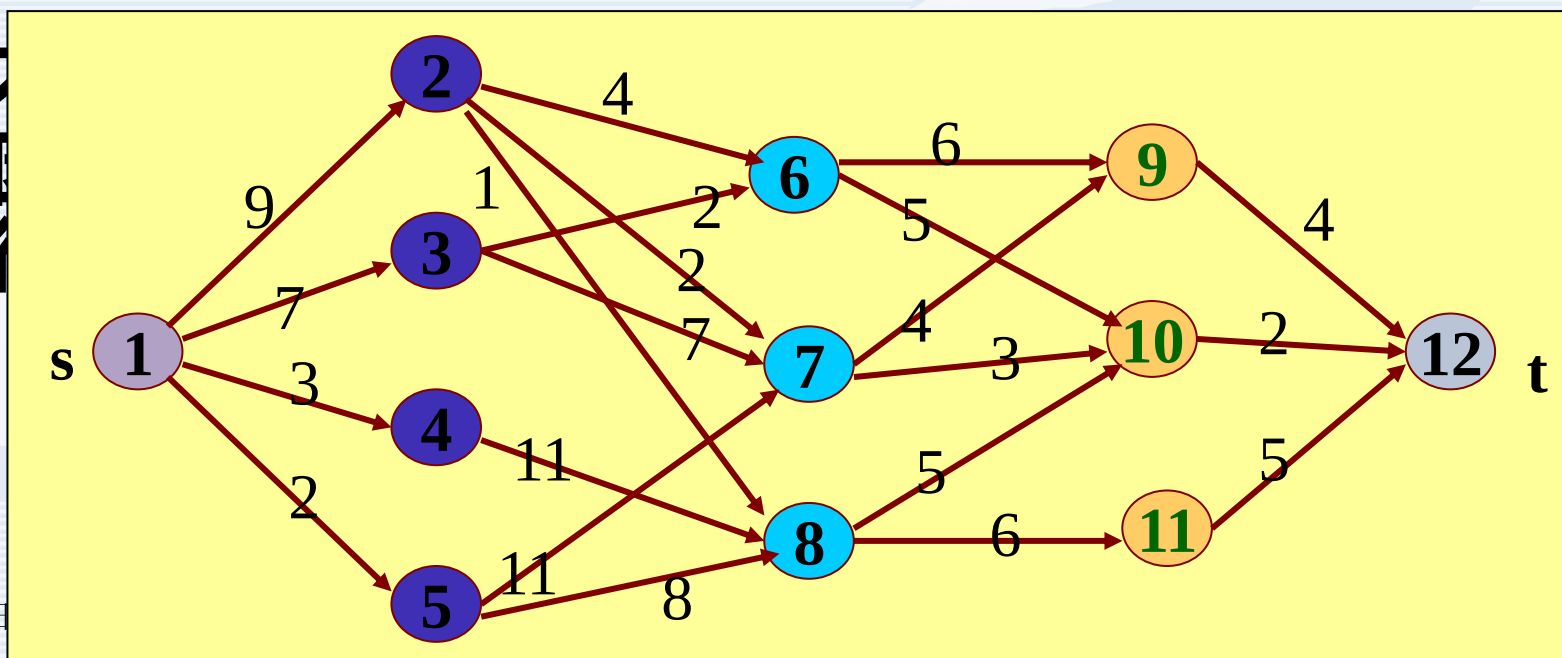
注意：同一个问题可以有多种方式刻画它的最优子结构，有些表示方法的求解速度更快 (空间占用小，问题的维度低)。

2 重叠子问题

- 递归算法求解问题时，每次产生的子问题并不总是新问题，有些子问题被反复计算多次。这种性质称为子问题的重叠性质。
- 动态规划算法，对每一个子问题只解一次，而后将其解保存在一个表格中，当再次需要解此子问题时，只是简单地用常数时间查看一下结果。
- 通常不同的子问题个数随问题的大小呈多项式增长。因此用动态规划算法只需要多项式时间，从而获得较高的解题效率。

2 重叠子问题

- 递归算法求解问题时，每次产生的子问题并不总是新问题，有些子问题被反复计算多次。这种性质称为子问题的重叠性质。
- 动态规划算法，对每一个子问题只解一次，而后将其解保存在一个表格中，当再次需要解此子问题时，只是简单地用常数时间查看一下结果。
- 通常长。而获得



◆ 动态规划算法和贪心选择算法、分治法的比较

- 动态规划法与分治法和贪心法类似，它们都是将问题实例归纳为更小的、相似的子问题，并通过求解子问题产生一个全局最优解。
- 其中贪心法的当前选择可能要依赖已经作出的所有选择，但不依赖于有待于做出的选择和子问题。因此贪心法自顶向下，一步一步地作出贪心选择。具有最优子结构，但不具有子问题重叠。
- 而分治法中的各个子问题是独立的（即不包含公共的子问题），因此一旦递归地求出各子问题的解后，便可自下而上地将子问题的解合并成问题的解。
- 但不足的是，如果当前选择可能要依赖子问题的解时，则难以通过局部的贪心策略达到全局最优解；如果各子问题是不独立的，则分治法要做许多不必要的工作，重复地解公共的子问题。

- 解决上述问题的办法是利用**动态规划**。该方法主要应用于最优化问题，这类问题会有多种可能的解，每个解都有一个值，而动态规划可找出其中最优(最大或最小)值的解。若存在若干个取最优值的解的话，它只取其中的一个(可能不唯一)。
- 在求解过程中，该方法也是通过**求解局部子问题的解达到全局最优解**，但与分治法和贪心法不同的是，动态规划允许这些子问题不独立，(亦即各子问题可包含公共的子问题)也允许其通过自身子问题的解作出选择，**该方法对每一个子问题只解一次，并将结果保存起来，避免每次碰到时都要重复计算。**

- 解决上述问题
- 应用于最佳化问题(即一个解都有一个或最小)值时,它只取其中最佳的一个解
- 在求解过程中,解达到全局最优

因此, 动态规划法所针对的问题有一个显著的特征, 即它所对应的子问题树中的子问题呈现大量的重复。动态规划法的关键就在于, 对于重复出现的子问题, 只在第一次遇到时加以求解, 并把答案保存起来, 让以后再遇到时直接引用, 不必重新求解。

动态规划允许这些子问题不是独立的, (亦即各子问题可包含公共的子问题), 也允许其通过自身子问题的解作出选择, 该法对每一个子问题只解一次, 并将结果保存起来, 避免每次碰到时都要重复计算。

◆如何用动态规划法求解

动态规划的设计方法对不同问题，有各具特色的表达方式。这是由于各种问题的性质不同，因此判定最佳解的条件也不相同。不存在一种万能的动态规划算法。但是，我们可以通过对若干有代表性的问题的动态规划算法的研究来学习这一设计方法。

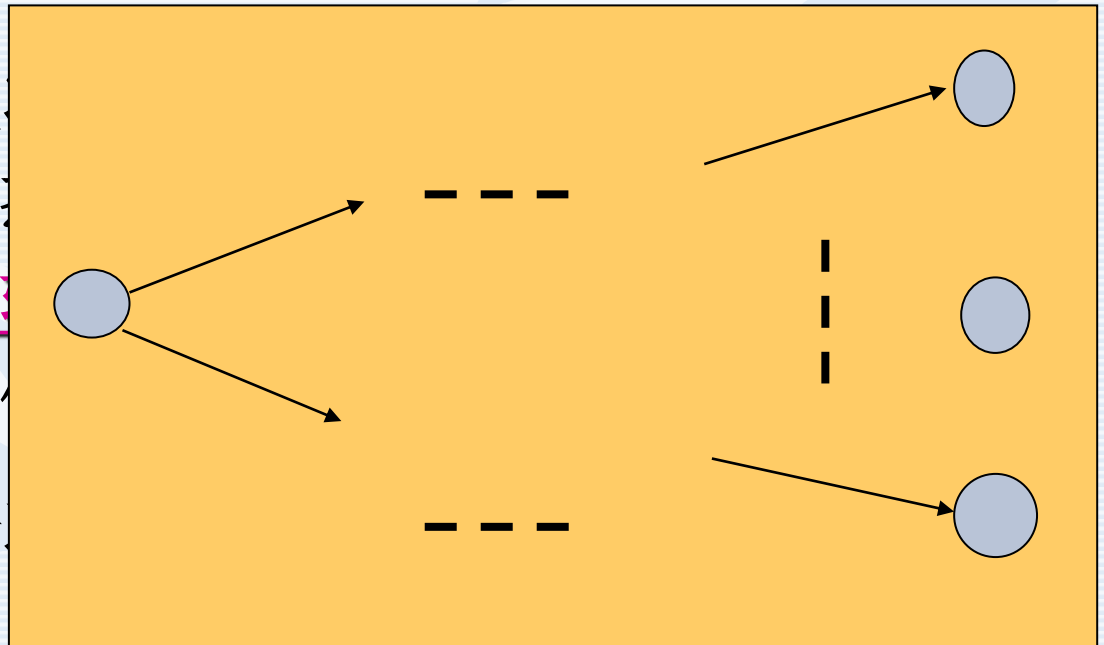
➤ **向前处理法（逆序法）**：从最后阶段开始，以逐步向前递推的方式列出求前一阶段决策值的递推关系式，即根据 $x_{i+1}, \dots, x_n$ 的那些最优决策序列来列出求取 x_i 决策值的关系式。

➤ **向后处理方法（顺序法）**：从初始阶段开始，逐步向后递推，根据 $x_1, \dots, x_{i-1}$ 的那些最优决策序列列出求 x_i 的递推关系式。

一般地说，两种解法在本质上没有什么不同，通常情况下，当初始状态给定时用向前处理解法较方便，当终止状态给定时用向后处理解法较方便。

但若初始状态虽已给定，然而终点状态较多，需比较到达不同终点状态的各个路径及指标函数值，以选取总效益最佳的终点状态时，以使用向后处理解法比较方便。

另外在初始状态和终止状态较多的情况下，第一步需处理的状态较多的方法为比较方便（可以自行试验他），灵活地选用这两种方法。



◆动态规划算法的基本步骤

设计一个动态规划算法，在应用中通常可按以下几个步骤进行：

- 1 找出最优解的性质，并刻画其结构特征。（划分）
 - 2 递归地定义最优解。（确定方程）
 - 3 以自底向上(由后向前推)的方式计算最优解。
 - 4 根据计算最优解时得到的信息，构造一个最优解。
- 表计算)

根据上述动态规划设计的步骤，也可得到大体解题框架如下：

- 1.初始化(边界条件)
- 2.for $i:=2$ to n (向后处理法) 或 for $i:=n-1$ to 1 (向前处理法)
对 i 阶段的每一个决策点求局部最优
- 3.确定和输出结束状态的值。

◆ 动态规划算法的时间复杂度特性

- 动态规划将原来具有指数级复杂度的搜索算法改进成了具有多项式时间的算法。其中的关键在于**解决冗余**，这是动态规划算法的根本目的。
- 动态规划实质上是一种以**空间换时间**的技术，它在实现的过程中，不得不存储产生过程中的各种状态，所以它的空间复杂度要大于其它的算法。
- 以旅行路线问题为例，与使用搜索算法来比较：

动态规划算法

搜索算法

时间： $T(n)=O(n^2)$

$T(n)=O(n!)$

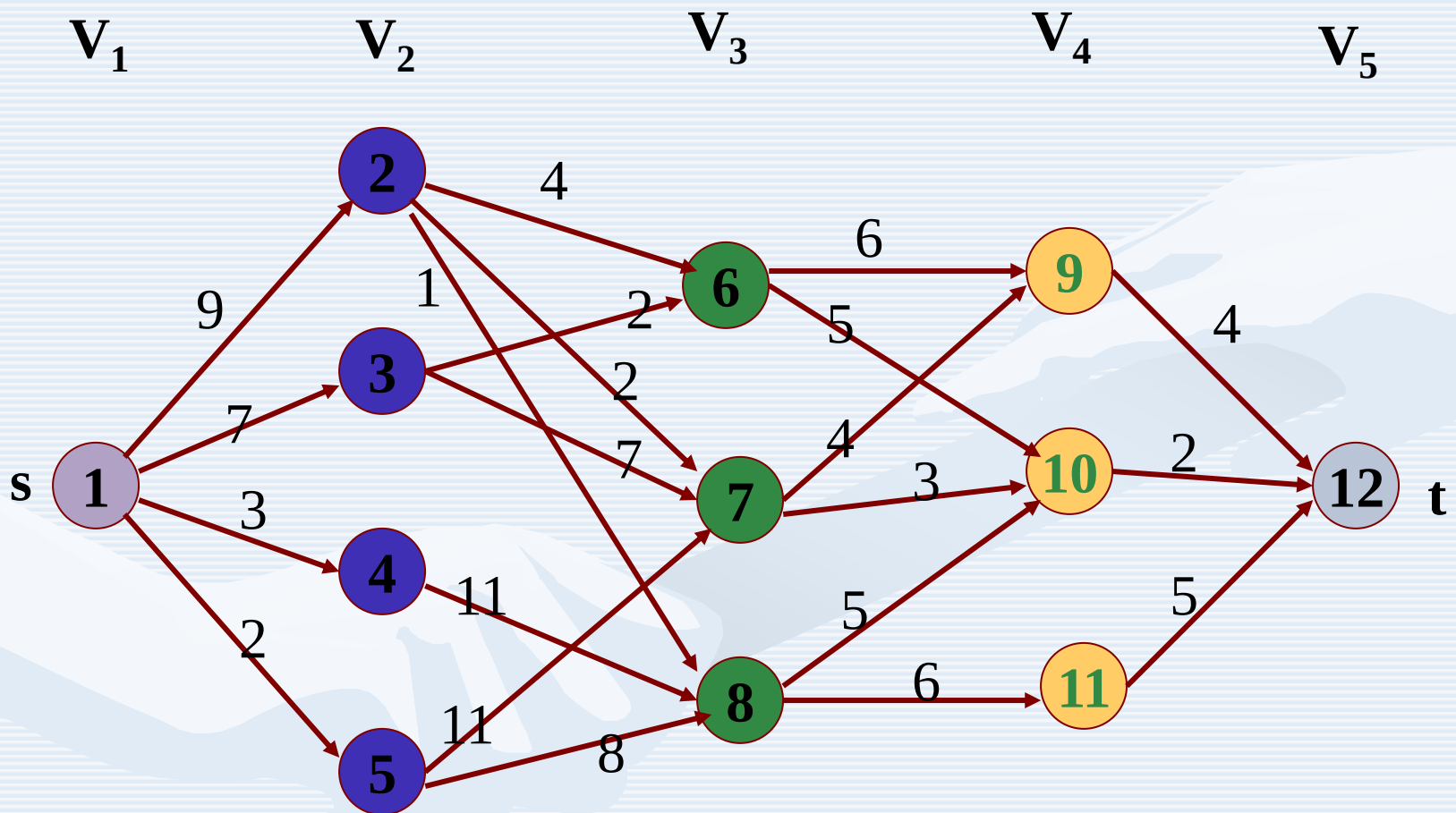
空间： $S(n)=O(n^2)$

$S(n)=O(n)$

搜索算法的 **$S(n)$** 反而优于动态规划算法。选择动态规划算法是因为动态规划算法在空间上可以承受，而搜索算法在时间上却无法承受，所以我们舍空间而取时间。

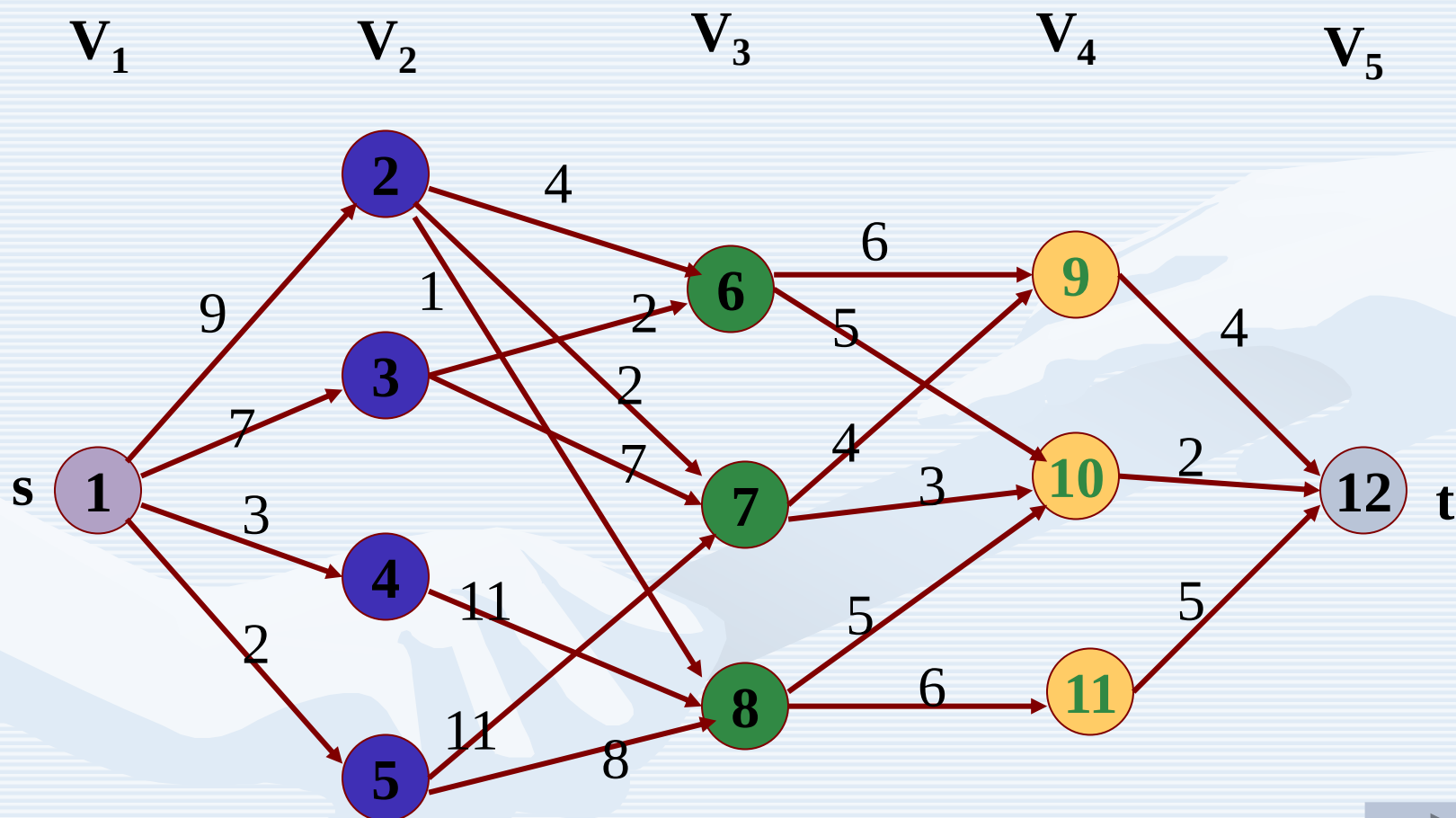
4.1.2 多段图问题

1. 多段图问题描述

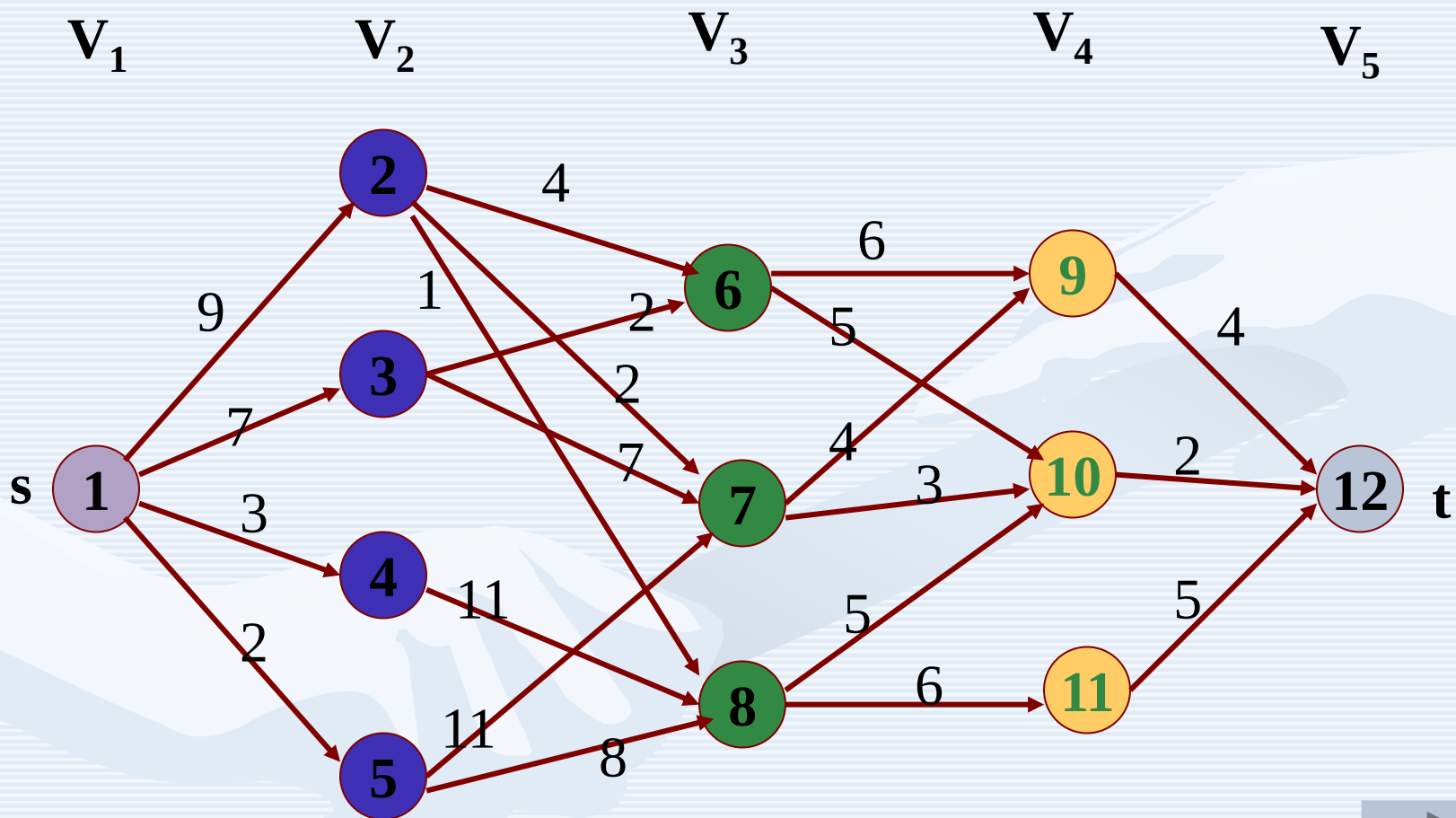


多段图 $G=(V, E)$ 是一个带权的有向图。
它具有如下特性：

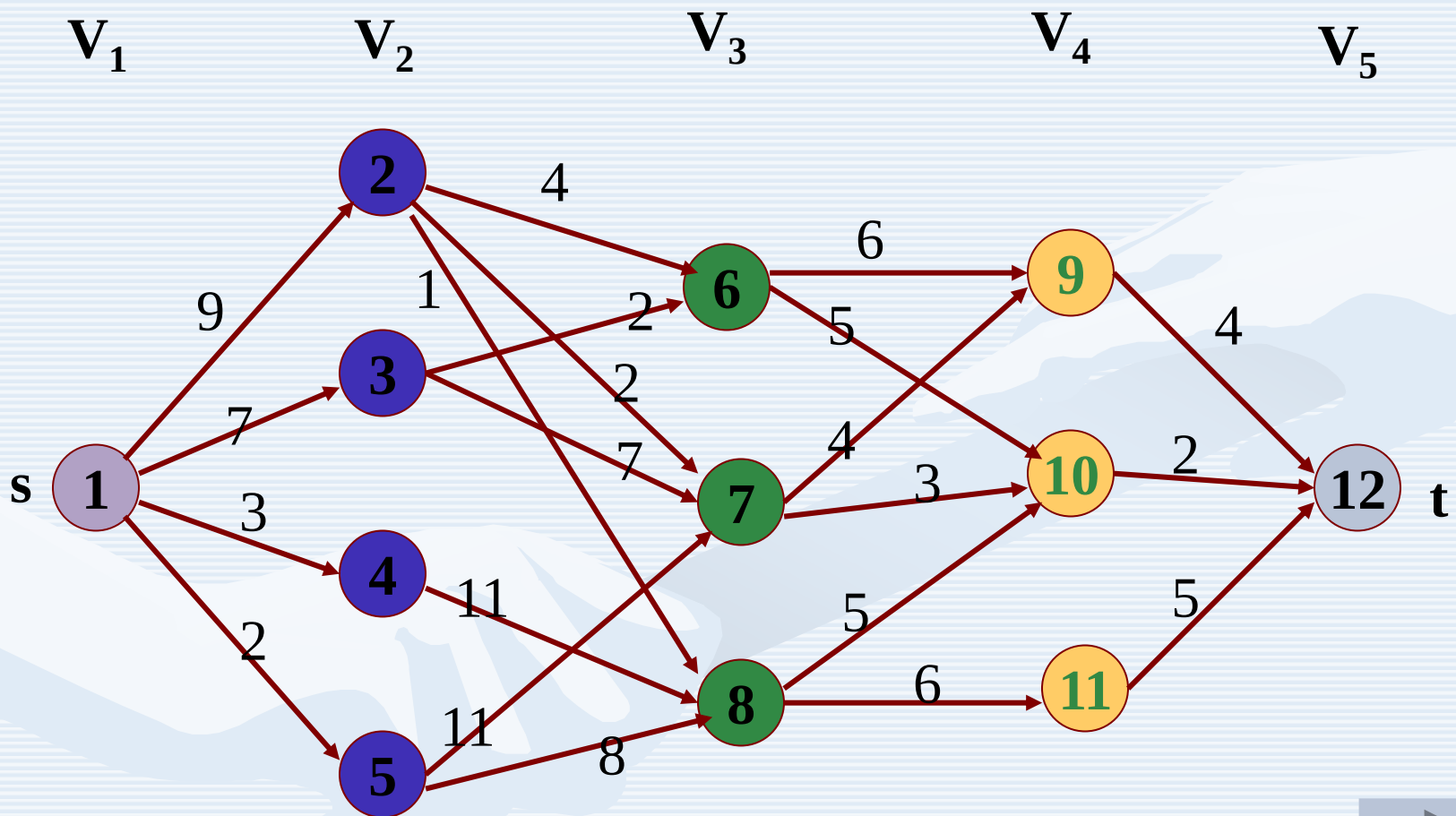
1. 多段图问题描述



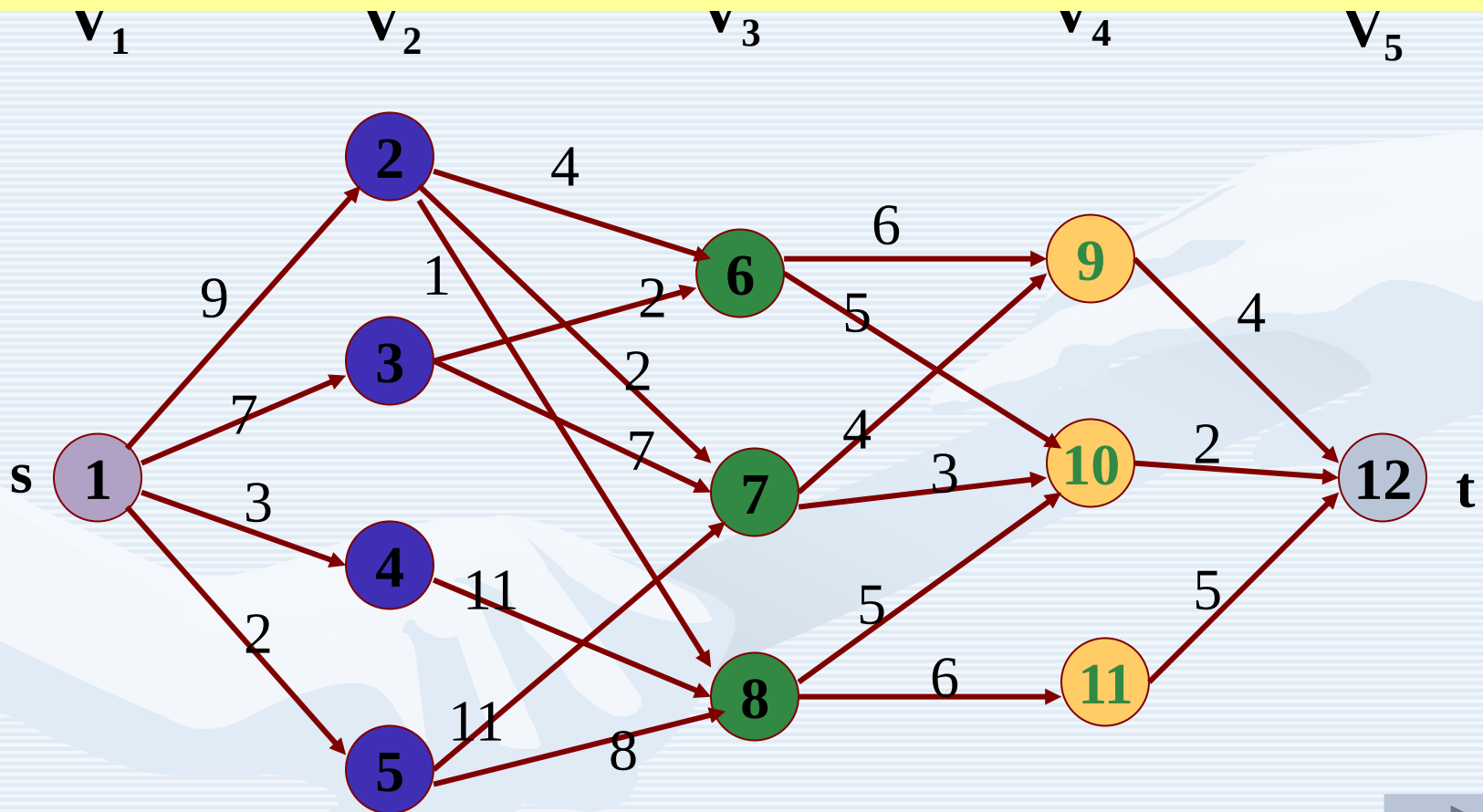
✓ 图中的结点被划分成 $k \geq 2$ 个不相交的集合 $V_i, 1 \leq i \leq k$, 其中 V_1 和 V_k 分别只有一个结点 s (源点) 和 t (汇点)。



✓图中所有的边 $\langle u, v \rangle$ 的始点和终点都在两个子集 V_i 和 V_{i+1} 中：即 $u \in V_i$ ， $v \in V_{i+1}$ ， $1 \leq i \leq k$ ，且每条边 $\langle u, v \rangle$ 均附有成本 $c(u, v)$ 。

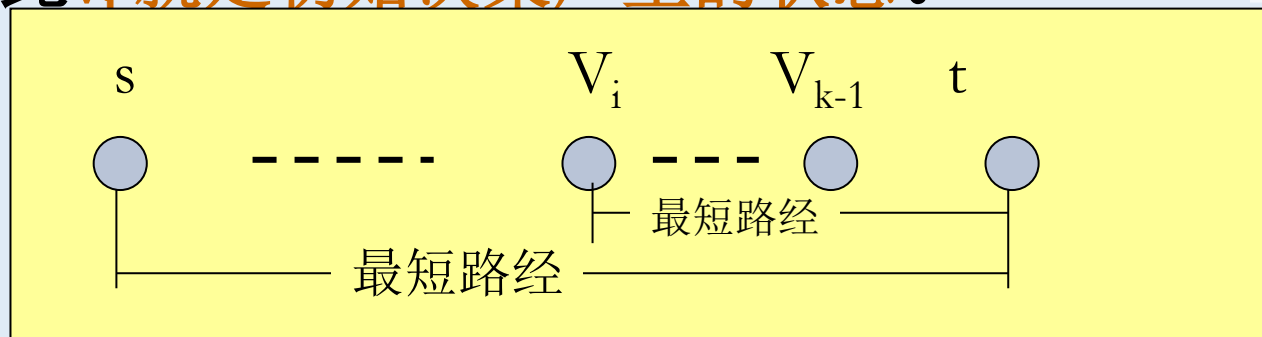


- ✓ 从s到t的一条路径成本是这条路径上边的成本和。
- ✓ 多段图问题(multistage graph problem)是求由s到t的最小成本路径。



2. 多段图问题最优化原理证明

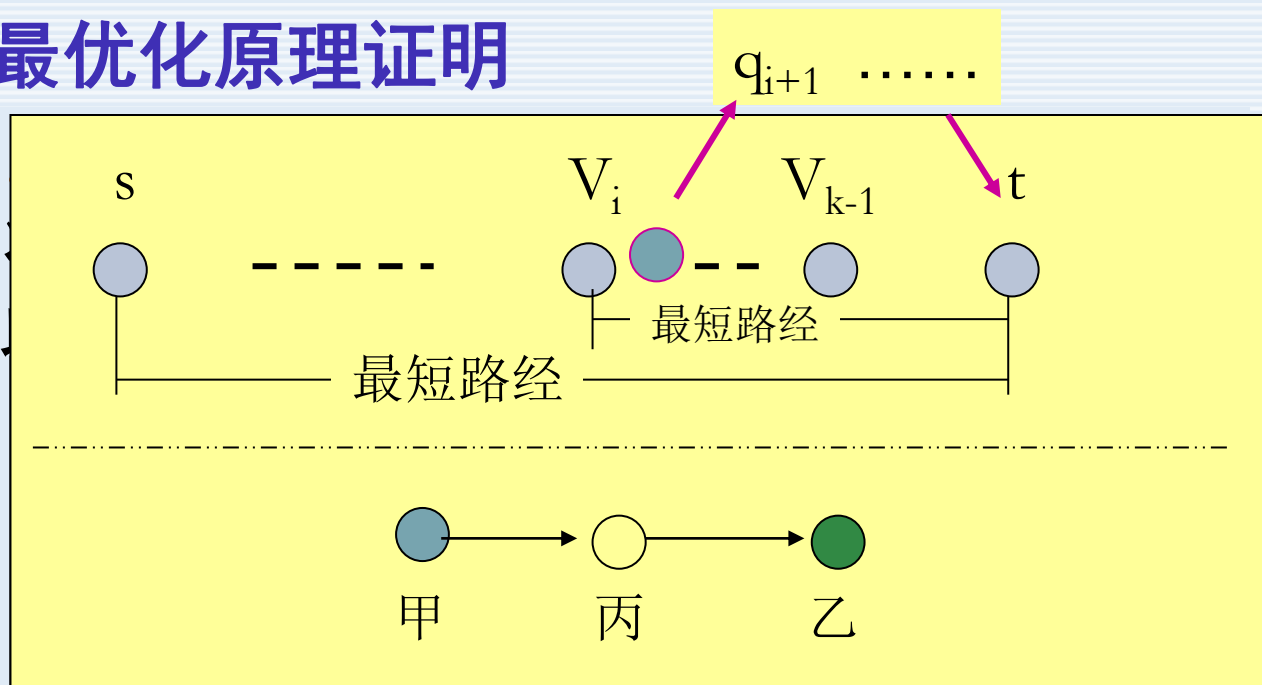
- 假设 $s, v_2, v_3, \dots, v_{k-1}, t$ 是一条由 s 到 t 的最短路径，还假设从源点 s (初始状态)开始，已作出了到顶点 v_i 的决策(初始决策)，因此 v_i 就是初始决策产生的状态。



- 若将 v_i 看成是原问题的子问题的初始状态，解这个子问题就是找一条由 v_i 到 t 的最短路径。这条最短路径一定是 $v_i, \dots, v_{k-1}, t$ 。否则，设 $v_i, q_{i+1}, \dots, q_{k-1}, t$ 是一条由 v_i 到 t 的更短的路径，则 $s, \dots, v_i, q_{i+1}, \dots, q_{k-1}, t$ 是一条由 s 到 t 的比 $s, \dots, v_i, \dots, v_{k-1}, t$ 更短的路径。与前面的假设矛盾。这说明多段图问题具有最优子结构性质，可以用动态规划的方法求最优解。

2. 多段图问题最优化原理证明

- 假设 $s, v_2, v_3, \dots$
设从源点 s (初始
(初始决策), 因



- 若将 v_i 看成是原问题的子问题的初始状态, 解这个子问题就是找一条由 v_i 到 t 的最短路径。这条最短路径一定是 $v_i, \dots, v_{k-1}, t$ 。否则, 设 $v_i, q_{i+1}, \dots, q_{k-1}, t$ 是一条由 v_i 到 t 的更短的路径, 则 $s, \dots, v_i, q_{i+1}, \dots, q_{k-1}, t$ 是一条由 s 到 t 的比 $s, \dots, v_i, \dots, v_{k-1}, t$ 更短的路径。与前面的假设矛盾。这说明多段图问题具有最优子结构性质, 可以用动态规划的方法求最优解。

4.2 多段图

4.2.1 多段图的向前处理算法

在实际应用中，许多问题的阶段划分并不明显，这时如果刻意地划分阶段法反而麻烦。一般来说，只要该问题可以划分成规模更小的子问题，并且原问题的最优解中包含了子问题的最优解（即满足最优化原理），则可以考虑用动态规划解决。

$$\text{COST}(i, j) = \min_{\substack{r \in V_{i+1} \\ \langle j, r \rangle \in E}} \{c(j, r) + \text{COST}(i+1, r)\}$$

- 因为，若 $\langle j, t \rangle \in E$ 成立，有 $\text{COST}(k-1, j) = c(j, t)$ ，若 $\langle j, t \rangle \in E$ 不成立，则有 $\text{COST}(k-1, j) = \infty$ ，所以可以通过如下步骤解公式（4.5），并求出 $\text{COST}(1, s)$ 。
- ◆ 首先对于所有 $j \in V_{k-2}$ ，计算 $\text{COST}(k-2, j)$ ，然后对所有的 $j \in V_{k-3}$ ，计算 $\text{COST}(k-3, j)$ 等等，最后计算出计算 $\text{COST}(1, s)$

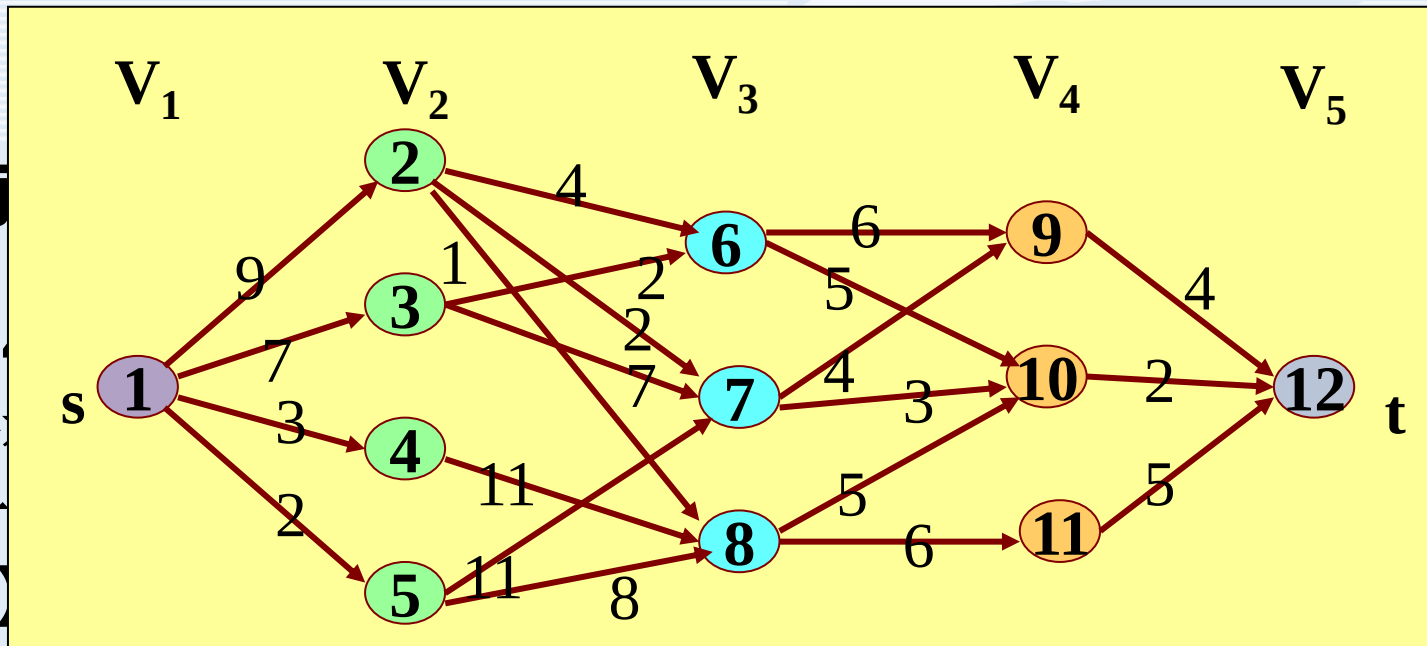
4.2 多段图

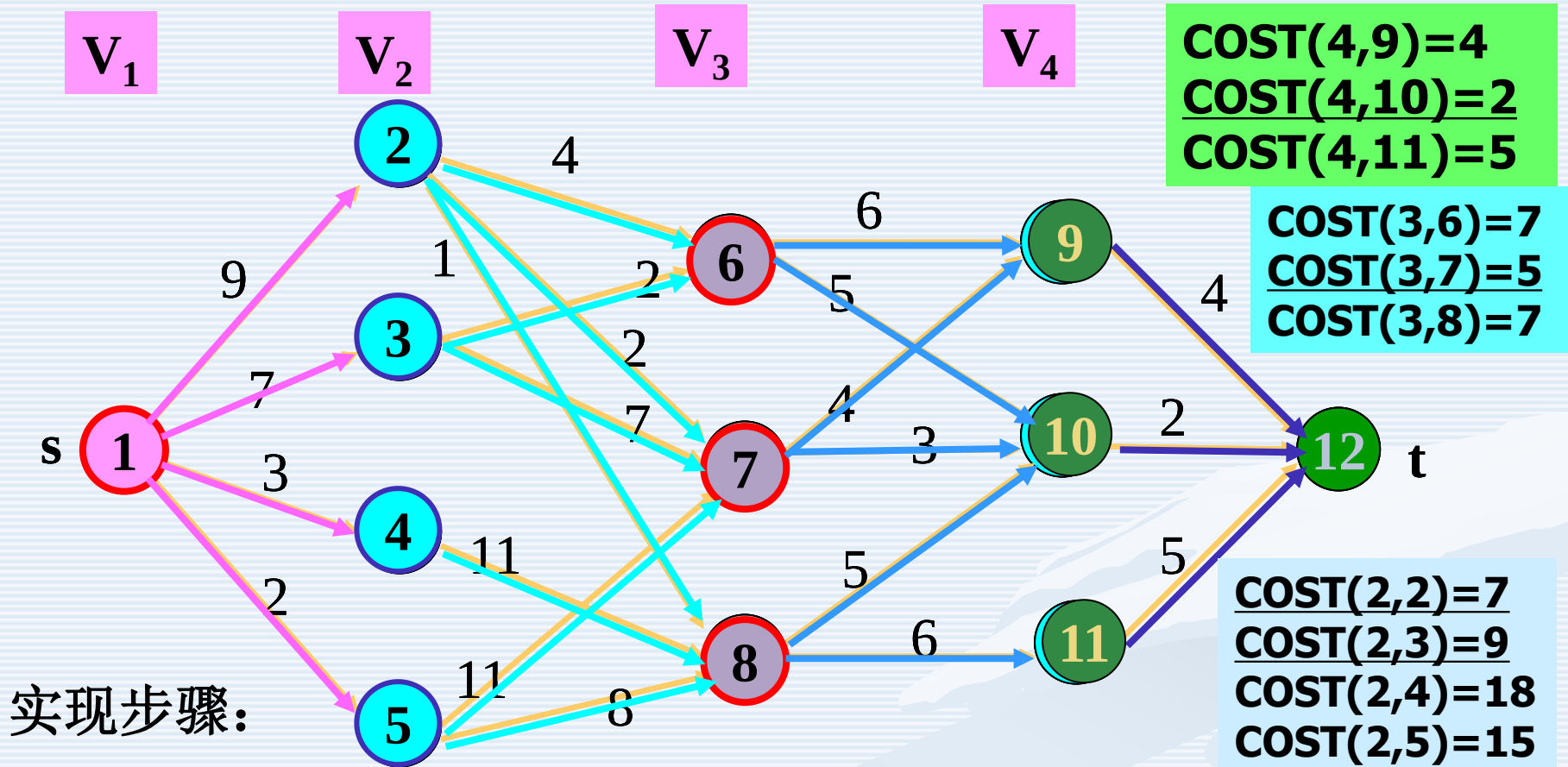
4.2.1 多段图的向前处理算法

在实际应用中，许多问题的阶段划分并不明显，这时如果刻意地划分阶段法反而麻烦。一般来说，只要该问题可以划分成规模更小的子问题，并且原问题的最优解中包含了子问题的最优解（即满足最优化原理），则可以考虑用动态规划解决。

$$\text{COST}(i, j) = \min \{c(j, r) + \text{COST}(i+1, r)\}$$

- 因为，若 $\langle j, r \rangle \in E$ 不成立，解式公式（
- ◆ 首先对于所 $j \in V_{k-3}$ ，计 $\text{COST}(1, s)$





• **COST(4,9)=4**

COST(2,2)=min{4+COST(3,6), 2+COST(3,7),

COST(1,1)=min{9+COST(2,2), 7+COST(2,3),
3+COST(2,4), 2+COST(2,5)}=16

COST(2,4)=18

COST(2,5)=15

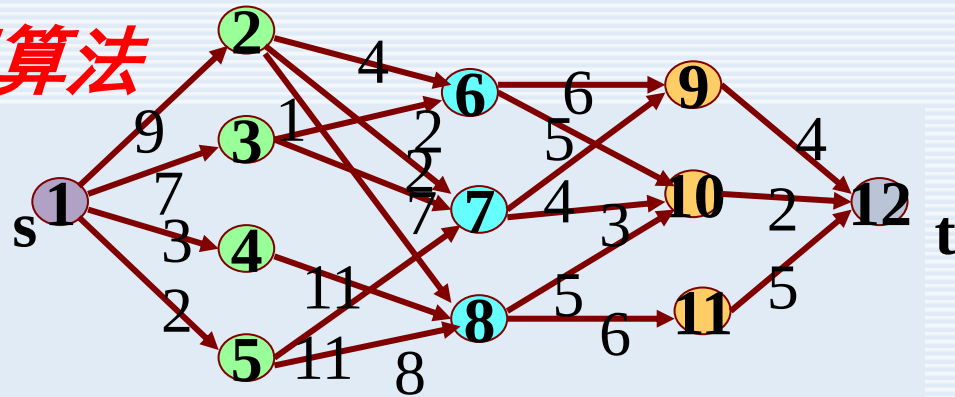
COST (i , j)=min {c (j , r)+COST (i+1,r)}

$r \in v_{i+1}$
 $\langle j, r \rangle \in E$

4.2.1 多段图的向前处理算法

例子中5段图的实现计算步骤:

- $\text{COST}(4,9)=4$
- $\text{COST}(4,10)=2$
- $\text{COST}(4,11)=5$
 - ◆ $\text{COST}(3,6)=\min\{6+\text{COST}(4,9), \underline{5+\text{COST}(4,10)}\}=7$
 - ◆ $\text{COST}(3,7)=\min\{4+\text{COST}(4,9), \underline{3+\text{COST}(4,10)}\}=5$
 - ◆ $\text{COST}(3,8)=\min\{\underline{5+\text{COST}(4,10)}, 6+\text{COST}(4,11)\}=7$
- $\text{COST}(2,2)=\min\{4+\text{COST}(3,6), \underline{2+\text{COST}(3,7)}, 1+\text{COST}(3,8)\}=7$
 - ◆ $\text{COST}(2,3)=\min\{\underline{2+\text{COST}(3,6)}, 7+\text{COST}(3,7)\}=9$
 - ◆ $\text{COST}(2,4)=18$
 - ◆ $\text{COST}(2,5)=15$
- $\text{COST}(1,1)=\min\{\underline{9+\text{COST}(2,2)}, \underline{7+\text{COST}(2,3)}, 3+\text{COST}(2,4), \underline{2+\text{COST}(2,5)}\}=16$



4.2.1 多段图的向前处理算法

Line procedure FGRAP(E,k,n,P)

real COST(n), integer D(n-1), P(k)

COST(n) $\leftarrow$ 0

for $j \leftarrow n-1$ to 1 by -1 do

设 r 是一个这样的结点, $\langle j, r \rangle \in E$ 且使 $c(j, r) + \text{COST}(r)$ 取小值

$\text{COST}(j) \leftarrow c(j, r) + \text{COST}(r)$

$D(j) \leftarrow r$

repeat

$P(1) \leftarrow 1; P(k) \leftarrow n$

for $j \leftarrow 2$ to $k-1$ do

$P(j) \leftarrow D(P(j-1))$

repeat

End FGRAPH

$\text{COST}(i, j) = \min \{c(j, r) + \text{COST}(i+1, r)\}$

$r \in V_{i+1}$
 $\langle j, r \rangle \in E$

$P(i, j)$ 是一条从 V_i 中的结点 j 到 t 的最小成本路径, $\text{COST}(i, j)$ 是这条路的成本。 $D(i, j)$ 记录了结点 j 的决策。

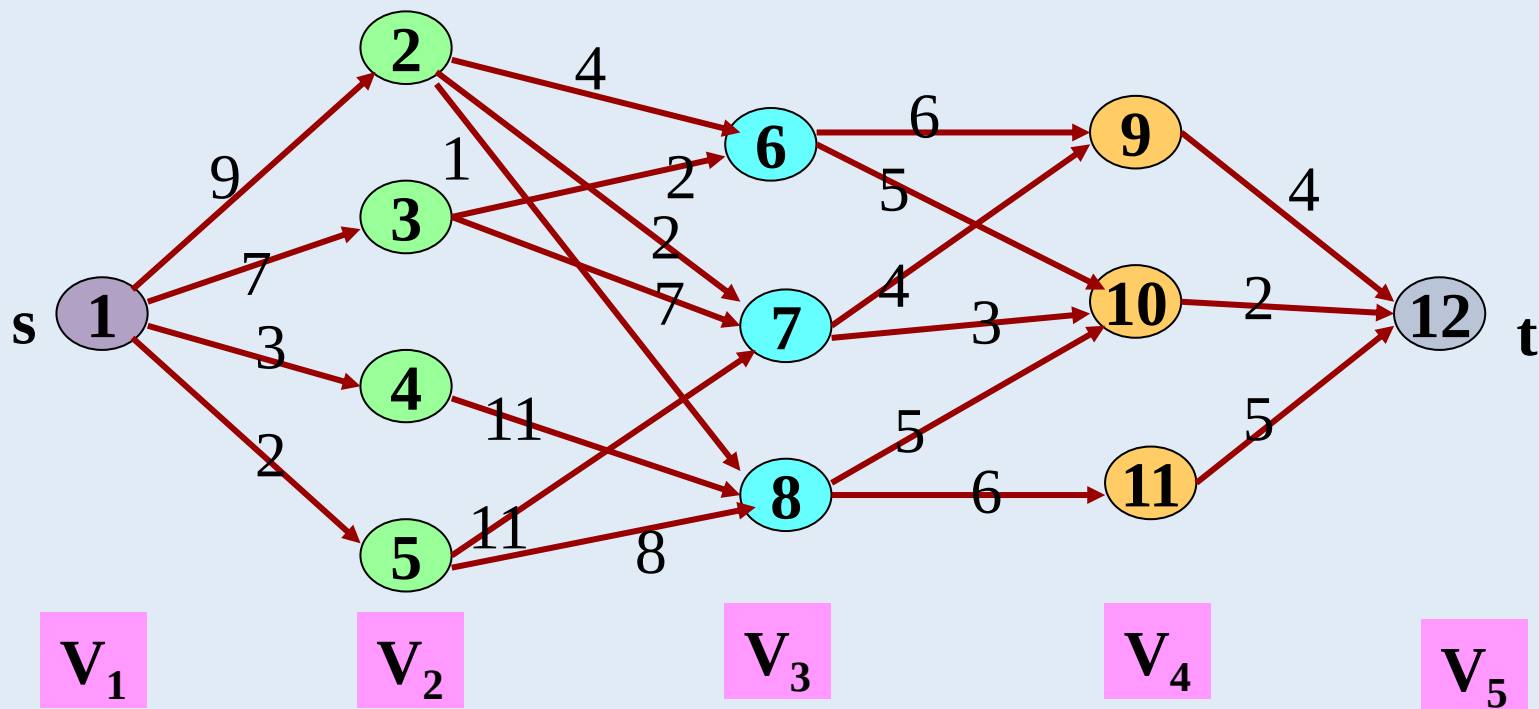
计算出 $\text{COST}(j)$ 的值, 并找出一条最小成本路径 $O(n+e)$

找出最小成本路径上的第 j 个结点 $O(k)$

当 V_{i+1} 中结点的编号均大于 V_i 中结点的编号, 可省略第一下标。次序按递减计算。

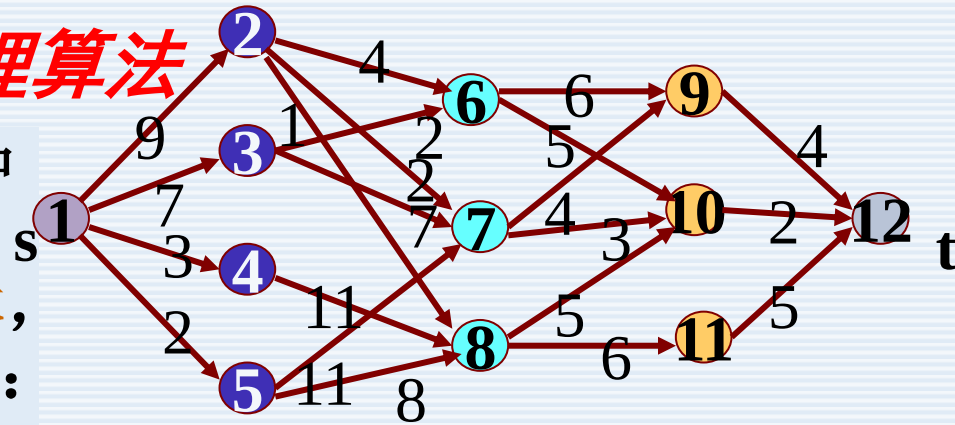
4.2.2 多段图的向后处理算法

- **向后处理方法**(backward approach)就是根据 $x_1, \dots, x_{i-1}$ 的那些最优决策序列列出求 x_i 的递推关系式。



4.2.2 多段图的向后处理算法

- 设 $BP(i, j)$ 是一条由源点 s 到 V_i 中结点 j 的最小成本路径，
 $BCOST(i, j)$ 表示 $BP(i, j)$ 的成本，
 由向后处理方法得到如下公式：



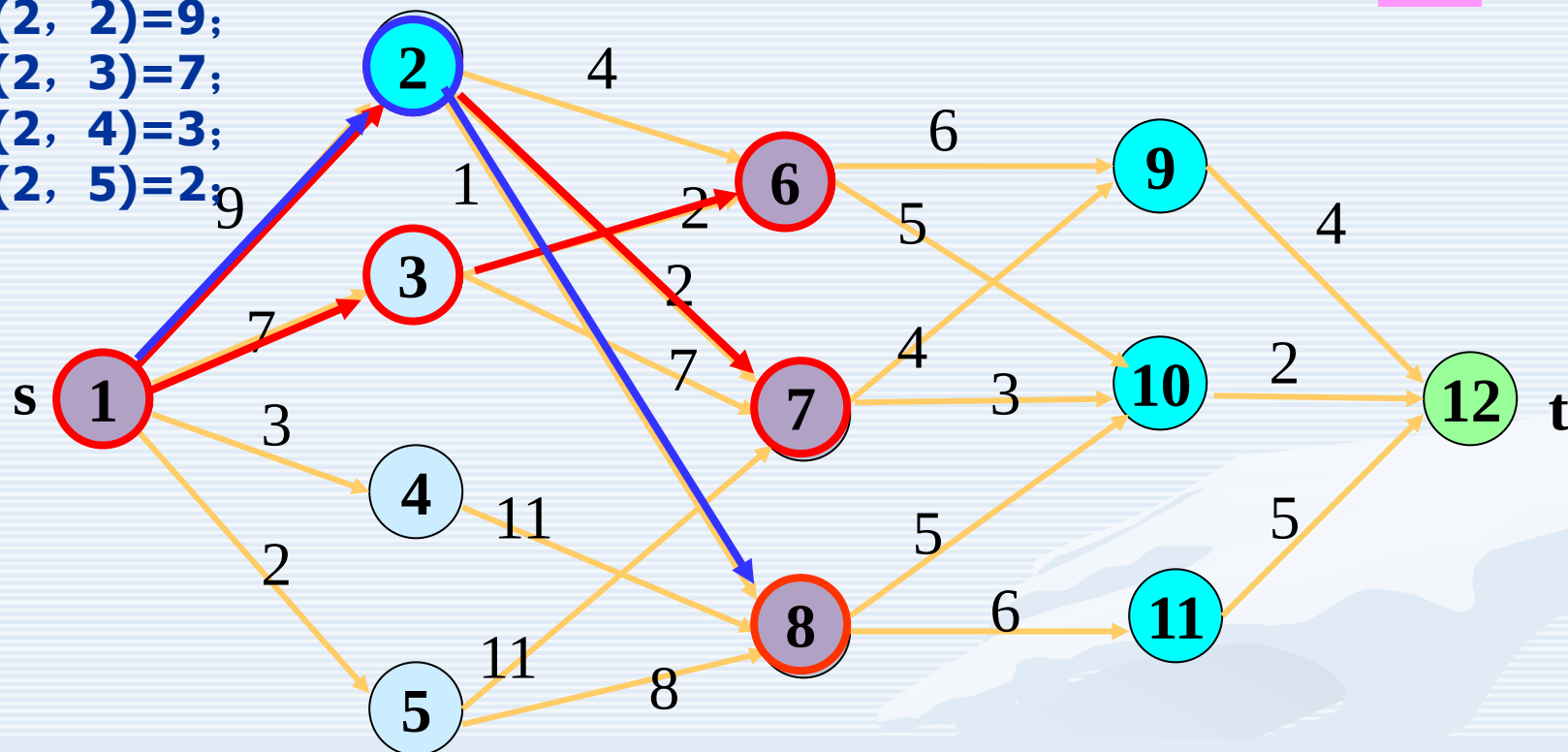
$$BCOST(i, j) = \min_{\substack{r \in V_{i-1} \\ \langle r, j \rangle \in E}} \{ BCOST(i-1, r) + c(r, j) \}$$

即由源点 s 到 V_i 中结点 j 的最小成本，等于由源点 s 到 V_{i-1} 中任一结点 r 的最小成本加上 V_{i-1} 中结点 r 到 V_i 中结点 j 的边长，所得的所有和中最小的那个值。

➤ 因为，若 $\langle 1, j \rangle \in E$ 成立， $BCOST(2, j) = c(1, j)$ ，若 $\langle 1, j \rangle \notin E$ 不成立，则有 $BCOST(2, j) = \infty$ ，所以可以通过如下步骤解式公式 (4.6)，首先对 $i=3$ 计算 $BCOST$ ，然后对 $i=4$ 计算 $BCOST$ 等，最后计算出 $BCOST(k, t)$ 。

➤ $BCOST(2, 2)=9$ ； $BCOST(2, 3)=7$ ；

➤ $BCOST(2, 4)=3$ ； $BCOST(2, 5)=2$ ；

V_1 V_2 V_3 V_4 V_5 $\text{BCOST}(2, 2)=9;$ $\text{BCOST}(2, 3)=7;$ $\text{BCOST}(2, 4)=3;$ $\text{BCOST}(2, 5)=2;$ 

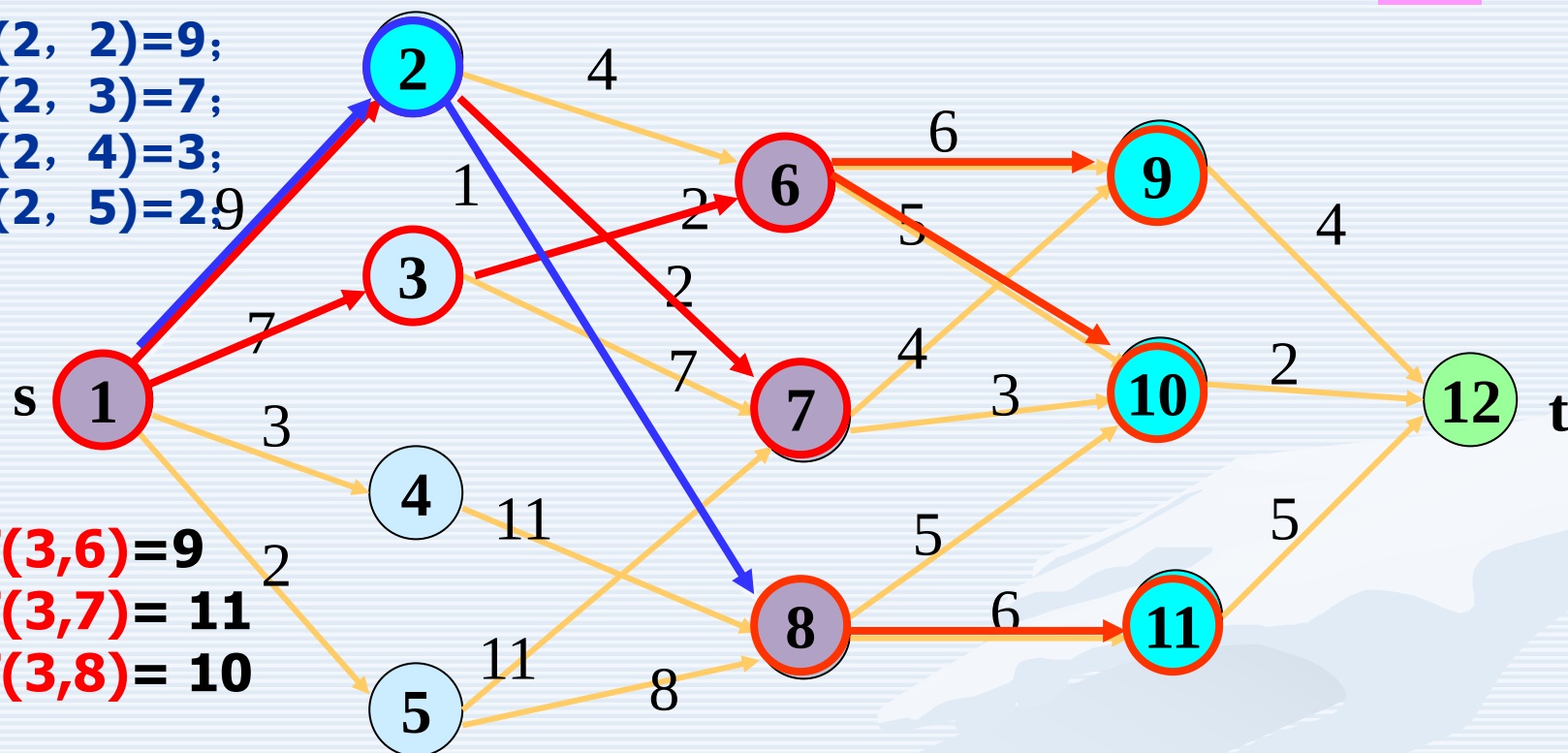
$\text{BCOST}(3,6) = \min\{\text{BCOST}(2,2)+4,$
 $\text{BCOST}(2,3)+2\} = 9$

$\text{BCOST}(3,7) = \min\{\text{BCOST}(2,2)+2,$
 $\text{BCOST}(2,3)+7, \text{BCOST}(2,5)+11\} = 11$

$\text{BCOST}(3,8) = \min\{\text{BCOST}(2,2)+1,$
 $\text{BCOST}(2,4)+11, \text{BCOST}(2,5)+8\} = 10$

V_1 V_2 V_3 V_4 V_5

$\text{BCOST}(2, 2)=9;$
 $\text{BCOST}(2, 3)=7;$
 $\text{BCOST}(2, 4)=3;$
 $\text{BCOST}(2, 5)=2$

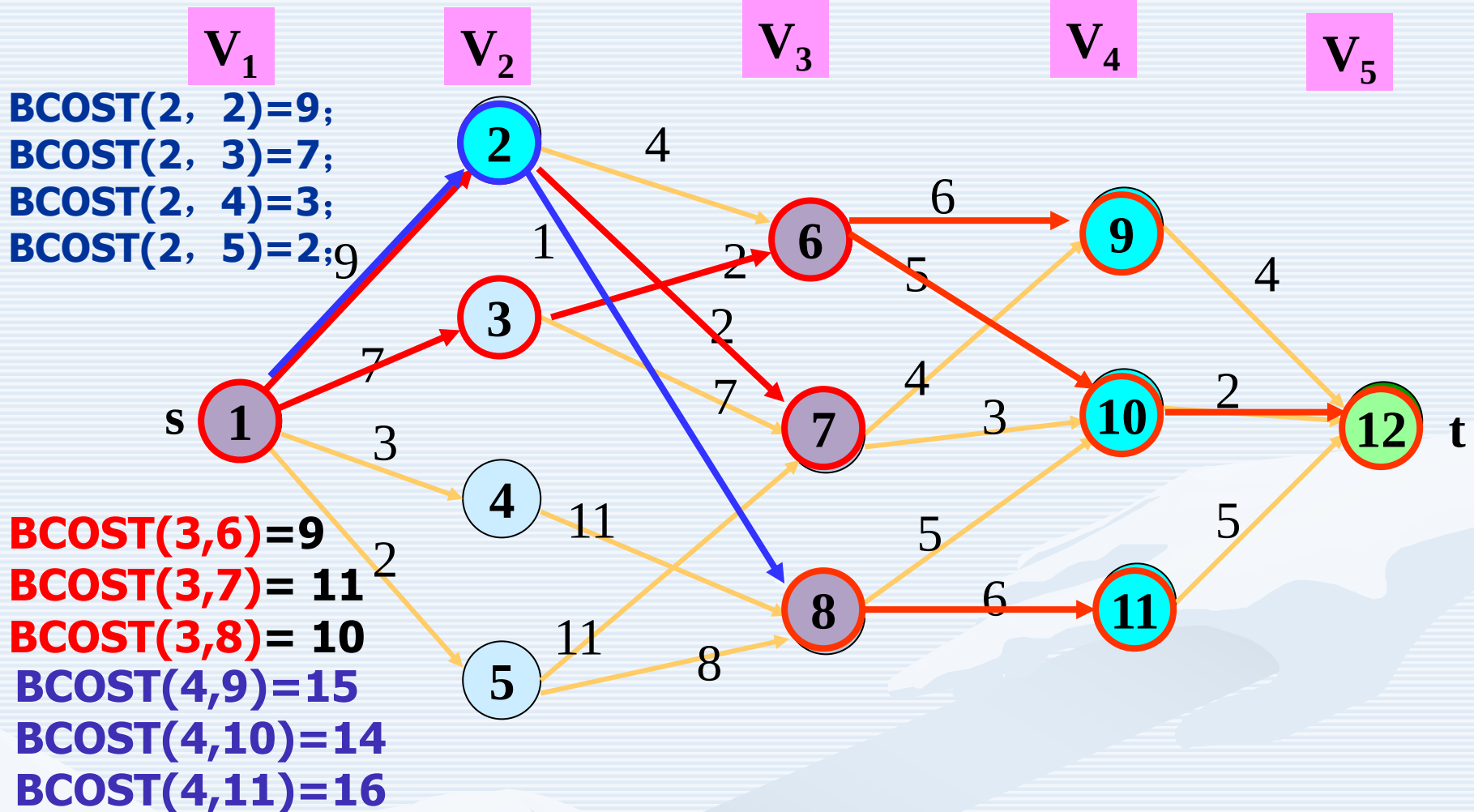


$\text{BCOST}(3,6)=9$
 $\text{BCOST}(3,7)=11$
 $\text{BCOST}(3,8)=10$

$\text{BCOST}(4,9)=\min\{\text{BCOST}(3,6)+6, \text{BCOST}(3,7)+4\}=15$

$\text{BCOST}(4,10)=\min\{\text{BCOST}(3,6)+5, \text{BCOST}(3,7)+3,$
 $\text{BCOST}(3,8)+5\}=14$

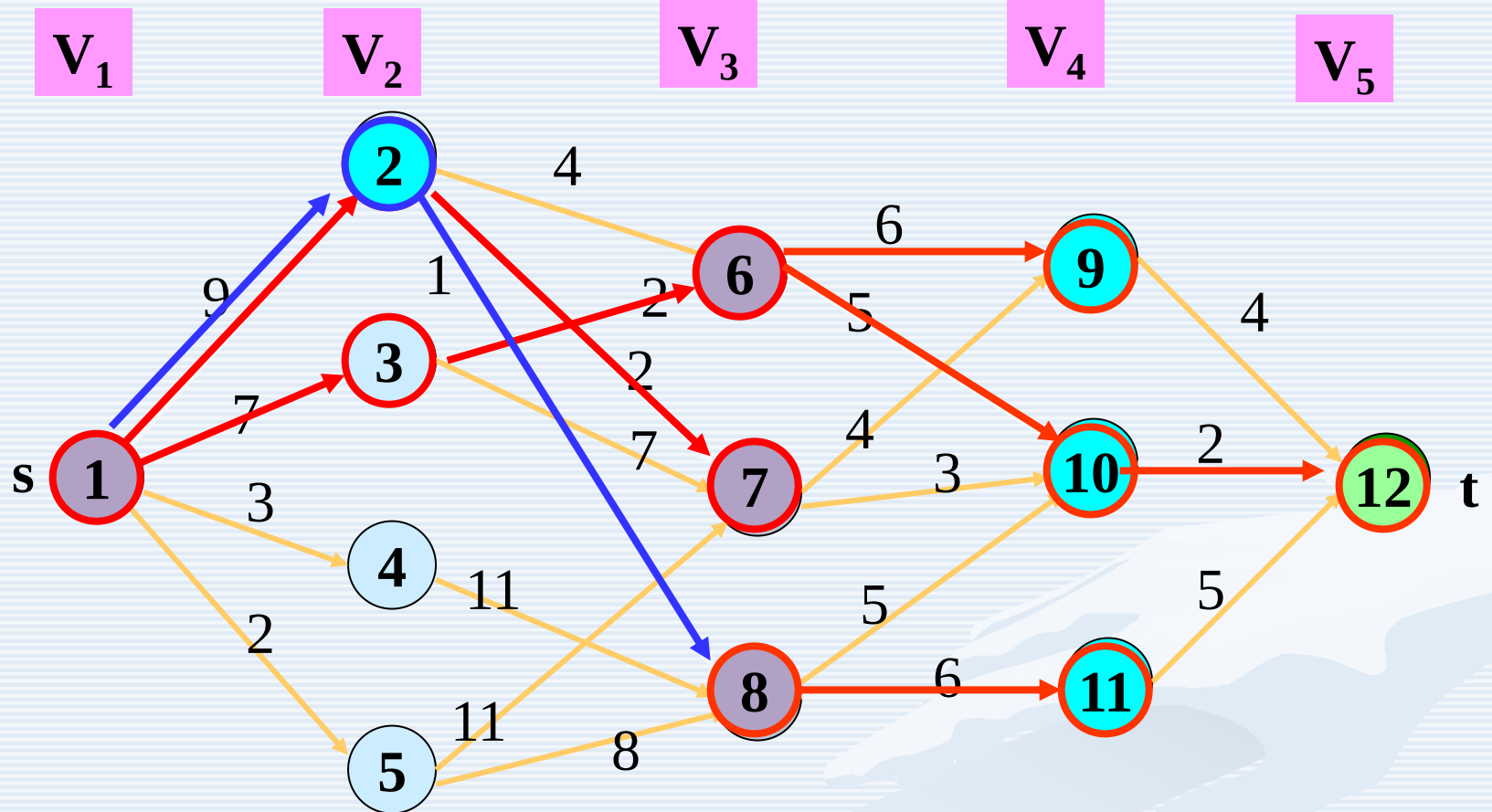
$\text{BCOST}(4,11)=16$



4.2.2 多段图的向后处理算法

- 求这条最小成本路径:

在计算每一个 $BCOST(i, j)$ 的同时, 记下每个状态(结点 j)所做出的决策(即, 使 $BCOST(i-1, j)+c(m, j)$ 取最小值的 m 值), 设为 $D(i, j)$, 则可容易地求出这条最小成本路径。



✨ 对于实例中的最小成本路径如下所示:

$D(3,6)=3$; $D(3,7)=2$; $D(3,8)=2$;

$D(4,9)=6$; $D(4,10)=6$; $D(4,11)=8$;

$D(5,12)=10$

4.2.2 多段图的向后处理算法

Line procedure BGRAP(E,k,n,P)

real BCOST(n),integer D(n-1),P(k),r,j,k,n

BCOST(1) $\leftarrow$ 0

for j $\leftarrow$ 2 to n do

 设r是一个这样的结点, $\langle r,j \rangle \in E$ 且使BCOST(r) + c(r,j)
 取小值

 BCOST(j) $\leftarrow$ BCOST(r)+ c(r,j)

 D(j) $\leftarrow$ r

repeat

 P(1) $\leftarrow$ 1;P(k) $\leftarrow$ n

 for j $\leftarrow$ k-1 to 2 by -1 do

 P(j) $\leftarrow$ D(P(j+1))

repeat

End BGRAPH

计算出BCOST(j)
的值, 并找出一
条最小成本路径

找出最小成本路径
上的第j个结点

时间复杂性问题

补4.3 资源分配问题

讨论将有限资源分配给若干个工程的问题。

4.3.1 问题描述

- 设资源总数为 a ，工程的个数为 n ，给每项工程分配的资源数额不同，所获得的利润也不同。在给定各工程的资源利润表和资源总数，怎样分配资源，才能获得最大利润呢？
- 这个问题的实质是要求能够给出资源总数 a 的一个划分 $x_1, x_2, \dots, x_n$ ， $0 \leq x_i \leq a$ ，且 $x_1 + x_2 + \dots + x_n \leq a$ ，使得利润 $G(a) = G_1(x_1) + G_2(x_2) + \dots + G_n(x_n)$ 最大。
- 这里 $G_i(x_i)$ 是把资源数 x_i 分配给第 i 项工程的最大利润， $1 \leq i \leq n$ ， $G_i(x)$ 是给定的已知函数。

4.3.2 解决方案

在 $G_i(x)$ 并非 x 的线性函数时，我们可以利用动态规划求解最佳分配方案。用 $f_i(x)$ 表示对前1项，前2项，...，前 i 项工程投资 x 时的最大利润，那么

$$\left\{ \begin{array}{l} f_1(x) = \max_{0 \leq x_1 \leq x \leq a} \{G_1(x_1)\} \\ f_2(x) = \max_{0 \leq x_2 \leq x \leq a} \{G_2(x_2) + f_1(x-x_2)\} \\ f_3(x) = \max_{0 \leq x_3 \leq x \leq a} \{G_3(x_3) + f_2(x-x_3)\} \\ \dots\dots\dots \\ f_n(x) = \max_{0 \leq x_n \leq x \leq a} \{G_n(x_n) + f_{n-1}(x-x_n)\} \end{array} \right.$$

这样，一个资源分配问题就转化为可用动态规划求解的多段决策过程。

4.3.3 实例

- 例 有资金6万元，拟投资A、B和C三种产品，利润函数 $G(x)$ 的值给定如下表。求最佳投资方案。

投资利润表 单位：万元

x_i	0	1	2	3	4	5	6
$G_A(x_1)$	0	1.2	1.5	1.85	2.4	2.8	3.3
$G_B(x_1)$	0	1.8	2.0	2.25	2.4	2.5	2.6
$G_C(x_1)$	0	1.3	1.9	2.2	2.45	2.7	3.0

①只投资一种产品，根据上表可知投入**A**获利最大。

$$f1(x) = \max \{G_A(x)\} = f1(6) = 3.3$$

②

此时为0：不给B投资，只给A投资，等于表1栏 $G_A(x)$

对第二种产品的各种投资额

x_i	0	1	2	3	4	5	6
$G_A(x_1)$	0	1.2	1.5	1.85	2.4	2.8	3.3
$G_B(x_1)$	0	1.8	2.0	2.25	2.4	2.5	2.6
$G_C(x_1)$	0	1.3	1.9	2.2	2.45	2.7	3.0

x_2 m	0	1	2					$f_2(m)$
0	0							0
1	1.2	1.8						1.8
2	1.5	3.0	2.0					3.0
3	1.85	3.3	3.2	2.25				3.3
4	2.4	3.65	3.5	3.45	2.4			3.65
5	2.8	4.2	3.85	3.75	3.6	2.5		4.2
6	3.3	4.6	4.4	4.1	3.9	3.7	2.6	4.6

总额 $a=4$ ，即B投资1万，而A投资3万时的利润。
即 $1.85+1.8=3.65$

对投资两种产品时的利润

所以，只对A和B两种产品投资时，产生的最大利润是4.6万元。

$$f_2(x) = \max\{G_B(x_2) + f_1(x - x_2)\} = G_B(1) + f_1(5) \\ = 1.8 + 2.8 = 4.6$$



$\begin{array}{c} x_2 \\ m \end{array}$	0	1	2	3	4	5	6	$f_2(m)$
0	0							0
1	1.2	1.8						1.8
2	1.5	3.0	2.0					3.0
3	1.85	3.3	3.2	2.25				3.3
4	2.4	3.65	3.5	3.45	2.4			3.65
5	2.8	4.2	3.85	3.75	3.6	2.5		4.2
6	3.3	4.6	4.4	4.1	3.9	3.7	2.6	4.6

所以，只对A和B两种产品投资时，产生的最大利润是4.6万元。

$$\begin{aligned}
 f_2(x) &= \max\{G_B(x_2) + f_1(x - x_2)\} = G_B(1) + f_1(5) \\
 &= 1.8 + 2.8 = 4.6
 \end{aligned}$$

③考虑投资A、B和C三种产品，根据
 $f_3(x) = \{G_C(x_3) + f_2(x - x_3)\}$

由表1和表2可以求得表3：

x_3	0	1	2	3	4	5	6
$G_C(x_3)$	0	1.3	1.9	2.2	2.45	2.7	3.0
$f_2(6-x_3)$	4.6	4.2	3.65	3.3	3.0	1.8	0
$f_3(6)$	4.6	5.5	5.55	5.5	5.45	4.5	3.0

可得

$$\begin{aligned}
 f_3(x) &= \max\{G_C(x_3) + f_2(m - x_3)\} = G_C(2) + f_2(4) \\
 &= G_C(2) + (G_B(1) + G_A(3)) \\
 &= 1.9 + (1.8 + 1.85) = 5.55
 \end{aligned}$$

由此得出获得最大利润的方案为：

对A产品投资3万元，对B产品投资1万元，对C产品投资2万元。

4.3.4 算法描述

- 设投资总额为 m ，工程个数为 n ，对工程 i 投资 j 的利润 $G(i,j)$ 已知， $1 \leq i \leq n$ ， $0 \leq j \leq m$ 。
- 用 AM 表示总利润，
- $x[i]$ 表示对工程 i 的资源分配结果， $1 \leq i \leq n$ ，
- $F[i,j]$ 表示 $f_i(j)$ 的最大利润，
- $P[i,j]$ 表示使 $f_i(j)$ 最大时，给第 i 项工程的资源数。

有限资源的最佳分配算法如下。

- ①生成表1—— $O(n)$
- ②生成表2、3—— $O(m^2 \cdot n)$
- ③其它—— $O(n)$

算法：有限资源最佳分配

这两句可计算出 $F[i,j]$ ，它是投资 j 给前 i 项工程的最佳方案。

计算对前 i 项工程投资的最大收益， $F[i,r]$ 是最佳方案。

```
PROCEDURE ALLOT ( $m, n, AM, x$ )  
1  for  $i \leftarrow 0$  to  $m$  do //初始化  
     $F[1,i] \leftarrow G[1,i]$ ;  $P[1,i] \leftarrow i$   
    repeat  
2   $W[1] \leftarrow \max\{F[1,0], F[1,1], \dots, F[1,m]\}$ ;  
3   $Q[1] \leftarrow k$ ; //  $k$ 是 $F[1,k]$  中最大值的下标  
4  for  $i \leftarrow 2$  to  $n$  do  
     $F[i,0] \leftarrow G[i,0] + F[i-1,0]$ ; //生成表2的第1列  
     $P[i,0] \leftarrow 0$ ;  
    for  $j \leftarrow 1$  to  $m$  do //生成表2的其他列,包括表3  
5      设 $k$ 是使得 $G[i,k] + F[i-1,j-k]$ 取最大值( $0 \leq k \leq j$ );  
6       $F[i,j] \leftarrow G[i,k] + F[i-1,j-k]$ ;
```

$G(i,j)$ 对工程 i 投资 j 的利润 $G(i,j)$ 已知， $1 \leq i \leq n$ ， $0 \leq j \leq m$ 。

$x[i]$ 表示对工程 i 的资源分配结果， $1 \leq i \leq n$ ，

$F[i,j]$ 表示 $f_i(j)$ 的最大利润，

$P[i,j]$ 表示使 $f_i(j)$ 最大时，给第 i 项工程的资源数。

```
10 AM ← max { W[1], W[2], ..., W[n] } ; //计算最大利润//
11 设s是使AM取最大的W[s]的下标; //找出对各项工程的投资数
12 if s ≠ n then
    for i ← s + 1 to n do x[i] ← 0 repeat
endif
13 x[s] ← P[s, Q[s]];
14 j ← m - x[s];
15 for i ← s - 1 to 1 by -1 do
    x[i] ← P[i, j]; j ← j - x[i]
repeat
end { ALLOT }
```

- 定理：算法ALLOT能在 $O(m^2 \cdot n)$ 的时间内找出资源的最佳

$G(i, j)$ 对工程*i*投资*j*的利润 $G(i, j)$ 已知, $1 \leq i \leq n$, $0 \leq j \leq m$ 。

$x[i]$ 表示对工程 *i*的资源分配结果, $1 \leq i \leq n$,

$F[i, j]$ 表示 $f_i(j)$ 的最大利润,

$P[i, j]$ 表示使 $f_i(j)$ 最大时, 给第*i*项工程的资源数。

4.1.3 0/1背包问题

1. 背包问题描述:

给定n种物品和一个容量为X的背包。物品i的重量是 w_i ，其价值为 p_i 。假定将物品i的放入背包产生 $p_i x_i$ 的效益, 应如何选择装入背包的物品，使得装入背包中物品的总价值最大？

背包问题中的 x_j 限定只能取0或1值，用KNAP(1,j,X)来表示这个问题

$$\text{极大化 } \sum_{1 \leq i \leq j} p_i x_i$$

效益值

$$\text{约束条件 } \sum_{1 \leq i \leq j} w_i x_i \leq X,$$

背包容量

$$x_i = 0 \text{ 或 } 1, \quad 1 \leq i \leq j.$$

不许分割

☛ 则0/1背包问题就是KNAP(1,n,M)

2. 0/1背包问题最优化原理证明

- 若 $y_1, y_2, \dots, y_n$ 是最优解, 则 $y_2, y_3, \dots, y_n$ 将是0/1背包问题的子问题

$$\max \sum_{2 \leq i \leq n} p_i x_i$$

$$\text{并且 } \sum_{2 \leq i \leq n} w_i x_i \leq M - w_1 \quad x_i \in \{0, 1\}, i = 2, \dots, n$$

$y_1=0$ 时, $y_2, y_3, \dots, y_n$ 必为相对于C的最优解;
 $y_1=1$ 时, $y_2, y_3, \dots, y_n$ 为相对于C- w_1 的最优解.

的最优解。不然, 若 $y_2', y_3', \dots, y_n'$ 是上式子问题的最优解, 且使得

$$\sum_{2 \leq i \leq n} p_i y_i' > \sum_{2 \leq i \leq n} p_i y_i$$

则 $y_1, y_2', y_3', \dots, y_n'$ 将是原问题的可行解, 并且使得

$$p_1 y_1 + \sum_{2 \leq i \leq n} p_i y_i' > \sum_{1 \leq i \leq n} p_i y_i$$

这与假设 $y_1, y_2, \dots, y_n$ 是最优解相矛盾。

因此, 0/1背包问题具有最优子结构性质得以证明, 可以用动态规划的方法求最优解。

4.4.2 0/1背包问题——解决方法

使用向后处理法求解。

- 根据问题描述，可通过作出变量 $x_1, x_2, \dots, x_n$ 的一个决策序列来得到它的解。
- 假定决策 x 的次序为 $x_1, x_2, \dots, x_n$ ，则在对 x_1 作出决策后，问题处于下列两种状态：
 - $x_1=0$ ，背包的剩余容量为 M ，没有产生任何效益
 - $x_1=1$ ，背包剩余容量为 $M-w_1$ ，效益值增加了 p_1
- 显然，剩下来对 $x_2, \dots, x_n$ 的决策相对于决策了 x_1 后所产生的问题状态应该是最优的，否则， $x_1, x_2, \dots, x_n$ 就不可能是最优的决策序列。

没有放入背包

放入背包

用背包剩余空间来决策的思想

4.4.2 0/1背包问题——解决方法

- 设 $f_i(y)$ 是 $\text{KNAP}(1,i,y)$ 最优解的值
- 则 $f_n(M)$ ($\text{KNAP}(1,n,M)$)可表示为:

$$f_n(M) = \max \{ f_{n-1}(M), f_{n-1}(M-w_n)+p_n \}$$

- 则对于任意的 $f_i(y)$ ($\text{KNAP}(1,i,y)$)可表示公式4.14为:

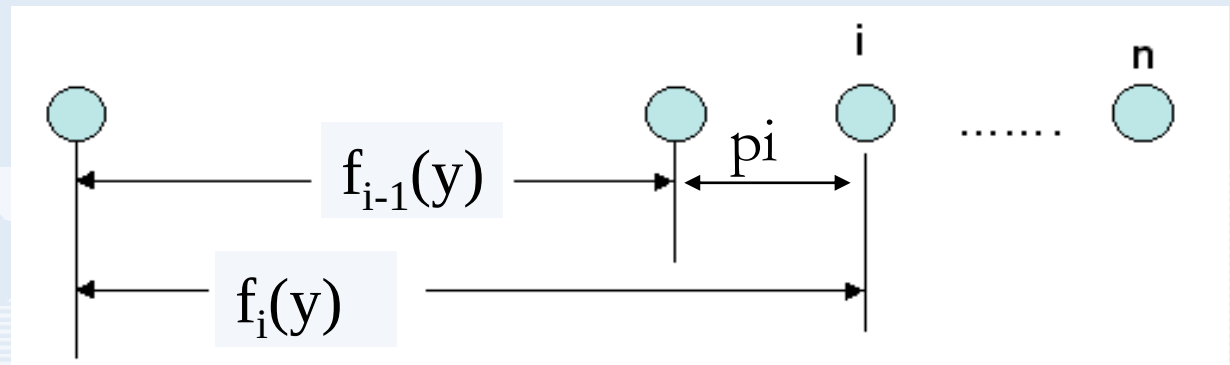
$$f_i(y) = \max \{ f_{i-1}(y), f_{i-1}(y-w_i)+p_i \}$$

这里 $i > 0$, $0 \leq y \leq M$,

不放入的决策

放入的决策

注意 y 的含义!



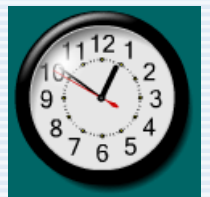
为了能由前向后递推而最后求解出 $f_n(M)$, 须从 $f_0(y)$ 开始

对于所有的 $y \geq 0$, 有 $f_0(y) = 0$;

当 $y < 0$ 时, 有 $f_0(y) = -\infty$

根据递推关系式, 马上可求出 $0 \leq y < w_1$ 和 $y \geq w_1$ 情况下的 $f_1(y)$ 的值, 以及 $f_2, \dots, f_n$ 的值。

y 是相对于背
包的剩余空间



4.4.3 0/1背包问题实例

$$f_i(y) = \max \{ f_{i-1}(y), f_{i-1}(y-w_i)+p_i \}$$

• 例: $n=3, M=6, (w_1, w_2, w_3)=(2, 3, 4), (p_1, p_2, p_3)=(1, 2, 5)$

$$\begin{aligned} f_3(M) = f_3(6) &= \max\{ f_2(6), f_2(6-w_3)+p_3 \} \\ &= \max\{ 3, f_1(2)+5 \} = 6 \end{aligned}$$

因此, 问题的解为 $(1, 0, 1)$

$$f_1(X) = \begin{cases} f_1(1) = \max\{ f_0(1), f_0(1-w_1)+p_1 \} = 0 \\ f_1(2) = \max\{ f_0(2), f_0(2-w_1)+p_1 \} = p_1 = 1 \\ \text{此时: } f_1(3) = f_1(4) = f_1(5) = f_1(6) = 1 \end{cases}$$

$$f_2(X) = \begin{cases} f_2(1) = \max\{ f_1(1), f_1(1-w_2)+p_2 \} = 0 \\ f_2(2) = \max\{ f_1(2), f_1(2-w_2)+p_2 \} = 1 \\ f_2(3) = f_2(4) = \max\{ f_1(3), f_1(3-w_2)+p_2 \} = 2 \\ f_2(5) = f_2(6) = \max\{ f_1(5), f_1(5-w_2)+p_2 \} = 3 \end{cases}$$

$$M=6, (w_1, w_2, w_3) = (2, 3, 4)$$

4.4.3 0/1背包问题实例

归纳:

$$f_0(X) = \begin{cases} -\infty & X < 0, \\ 0 & X \geq 0, \end{cases}$$

$$f_1(X) = \begin{cases} -\infty & X < 0, \\ \max \{ 0, -\infty + 1 \} = 0 & 0 \leq X < 2, \\ \max \{ 0, 0 + 1 \} = 1 & X \geq 2. \end{cases}$$

$$f_2(X) = \begin{cases} -\infty & X < 0, \\ 0 & 0 \leq X < 2, \\ 1 & 2 \leq X < 3, \\ \max \{ 1, 0 + 2 \} = 2 & 3 \leq X < 5, \\ \max \{ 1, 1 + 2 \} = 3 & X \geq 5, \end{cases}$$

$$f_3(M) = \max \{ 3, 1 + 5 \} = 6.$$



4.4.3 0/1背包问题实例

$f_2(y)=1$ 是在不放入得出的
所以 $x_2=1$

- 可通过检测 f_i 的取值情况可以确定得最优解的最优决策序列
 - $f_3(M)=6$ 是在 $x_3=1$ 的情况下取得的, 因此 $x_3=1$
 - $f_3(M)-p_3=1$, $f_2(y)$ 和 $f_1(y)$ 都可取1, 则 $x_2=0$
 - f_0 不能取值1, 故 $x_1=1$
 - 于是最优决策序列为 $(x_1, x_2, x_3)=(1, 0, 1)$
- 也可通过图解法得到
 - $f_{i-1}(y-w_i)+p_i$ 的图像可由 $f_{i-1}(y)$ 在 x 轴上右移 w_i 个单位, 然后上移 p_i 个单位得到
 - $f_i(X)$ 的图像由 $f_{i-1}(y)$ 和 $f_{i-1}(y-w_i)+p_i$ 的函数曲线按 y 相同时取最大值的规则归并而成
- 例: $n=3, M=6, (w_1, w_2, w_3)=(2, 3, 4), (p_1, p_2, p_3)=(1, 2, 5)$
根据公式4.14递推求解:

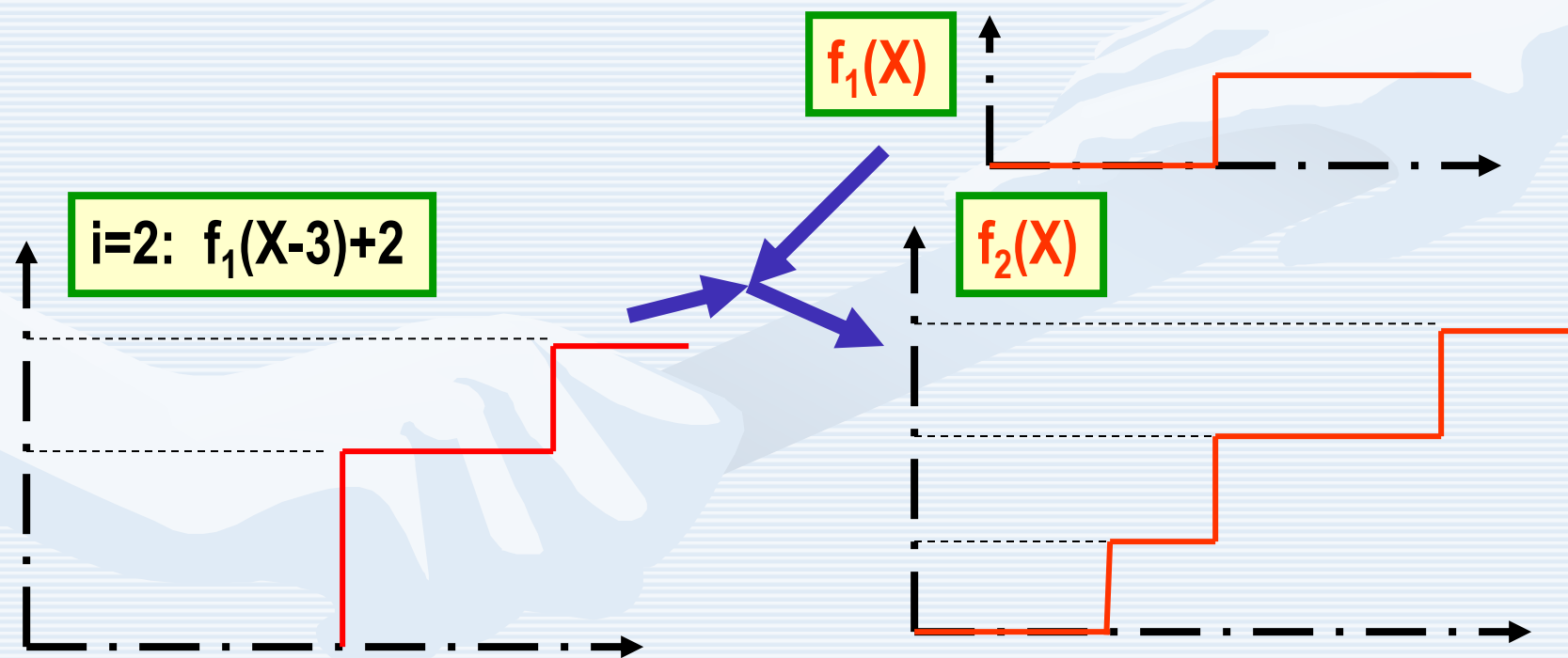
✓当 $X < 0$ 时, $f_2(X) = -\infty$

✓当 $0 \leq X < 2$ 时, $f_2(X) = \max\{f_1(X), f_1(X-3)+2\} = \max\{0, -\infty+2\} = 0$

✓当 $2 \leq X < 3$ 时, $f_2(X) = \max\{f_1(X), f_1(X-3)+2\} = \max\{1, -\infty+2\} = 1$

✓当 $3 \leq X < 5$ 时, $f_2(X) = \max\{f_1(X), f_1(X-3)+2\} = \max\{1, 0+2\} = 2$

✓当 $5 \leq X$ 时, $f_2(X) = \max\{f_1(X), f_1(X-3)+2\} = \max\{1, 1+2\} = 3$



- 当 $X < 0$ 时, $f_3(X) = -\infty$
- 当 $0 \leq X < 2$ 时, $f_3(X) = \max\{f_2(X), f_2(X-4)+5\} = \max\{0, -\infty + 5\} = 0$
- 当 $2 \leq X < 3$ 时, $f_3(X) = \max\{f_2(X), f_2(X-4)+5\} = \max\{1, -\infty + 5\} = 1$
- 当 $3 \leq X < 4$ 时, $f_3(X) = \max\{f_2(X), f_2(X-4)+5\} = \max\{2, -\infty + 5\} = 2$
- 当 $4 \leq X < 5$ 时, $f_3(X) = \max\{f_2(X), f_2(X-4)+5\} = \max\{2, 0 + 5\} = 5$
- 当 $5 \leq X < 6$ 时, $f_3(X) = \max\{f_2(X), f_2(X-4)+5\} = \max\{3, 0 + 5\} = 5$
- 当 $6 \leq X < 7$ 时, $f_3(X) = \max\{f_2(X), f_2(X-4)+5\} = \max\{3, 1 + 5\} = 6$
- 当 $7 \leq X < 9$ 时, $f_3(X) = \max\{f_2(X), f_2(X-4)+5\} = \max\{3, 2 + 5\} = 7$
- 当 $9 \leq X$ 时, $f_3(X) = \max\{f_2(X), f_2(X-4)+5\} = \max\{3, 3 + 5\} = 8$

